Academic year
2025 - 2026

Exam Programming: PINN
Using Physics Informed Neural Networks to solve nonlinear Partial Differential Equations

Des De Borger

University of Antwerp
Faculty of Science

Contents

1 Introduction

2 Damped Harmonic Oscillator

In this chapter, we will apply PINNs to solve the damped harmonic oscillator, a secondary order ODE which describes oscillatory motion with friction. The code for this chapter can be found in the `Damped_Oscillator` directory.

2.1 Analytic solution

The central equation we are trying to solve is given by:

$$my''(t) + cy'(t) + ky(t) = 0 \tag{2.1}$$

This is a second order linear ordinary differential equation. The formal solution of this equation can be found by substituting $y(t) = e^{rt}$ and solving the characteristic equation:

$$mr^2 + cr + k = 0 \tag{2.2}$$

The roots of this equation are given by:

$$r_{1,2} = \frac{-c \pm \sqrt{c^2 - 4mk}}{2m} \tag{2.3}$$

It can be useful to introduce the parameters $\omega_0 = \sqrt{k/m}$ and $\zeta = c/(2\sqrt{mk})$, in which case the roots of the equation become:

$$r_{1,2} = -\zeta\omega_0 \pm \omega_0\sqrt{\zeta^2 - 1} \tag{2.4}$$

We can distinguish three cases based on the value of the discriminant $D = c^2 - 4mk$ and ζ. Figures 2.1 to 2.3 illustrate these cases.

Overdamped If $D > 0$ ($\zeta > 1$), the roots are real and the solution is given by:

$$y(t) = Ae^{r_1 t} + Be^{r_2 t} \tag{2.5}$$

Where $r_1,\ r_2 < 0$ are real and A and B are constants determined by the initial conditions $y(0) = y_0$ and $y'(0) = dy_0$:

$$\begin{aligned} A &= \frac{dy_0 - r_2 y_0}{r_1 - r_2} \\ B &= \frac{-dy_0 + r_1 y_0}{r_1 - r_2} \end{aligned} \tag{2.6}$$

Critically damped if $D = 0$ ($\zeta = 1$), the roots $r_1 = r_2 = r = -\zeta\omega_0$ are negative, real and equal. A second linearly independent solution is then given by te^{rt}, so the generalsolution is given by:

$$y(t) = (A + Bt)e^{rt} \tag{2.7}$$

The constants A and B are again determined by the initial conditions:

$$\begin{aligned} A &= y_0 \\ B &= dy_0 - ry_0 \end{aligned} \tag{2.8}$$

Underdamped if $D < 0$ ($\zeta < 1$), the roots r_1 and r_2 are complex conjugates, whose complex parts are conventionally expressed as $\omega_d = \omega_0\sqrt{1-\zeta^2}$, so that the roots can be expressed as $r_{1,2} = -\zeta\omega_0 \pm i\omega_d$. The solution is then given by:

$$y(t) = e^{-\zeta\omega_0 t}(A\cos(\omega_d t) + B\sin(\omega_d t)) \tag{2.9}$$

The constants A and B are again determined by the initial conditions:

$$\begin{aligned} A &= y_0 \\ B &= \frac{dy_0 + \zeta\omega_0 y_0}{\omega_d} \end{aligned} \tag{2.10}$$

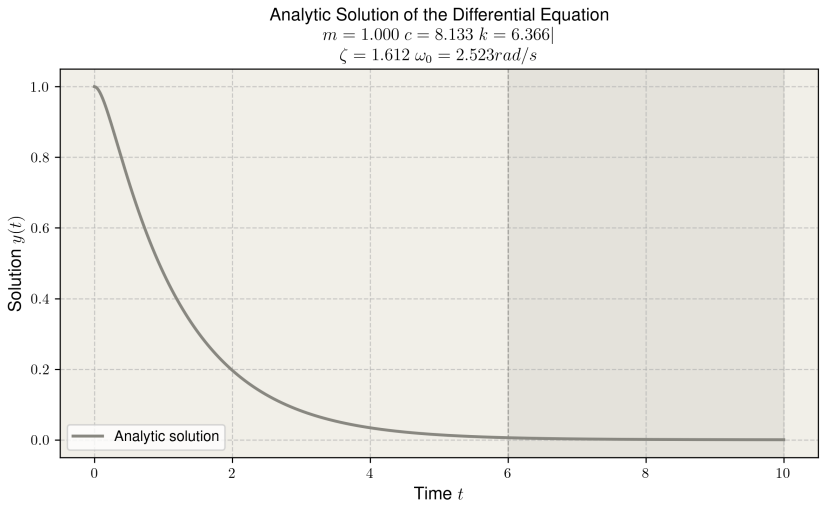


Figure 2.1: Analytic solution for the overdamped harmonic oscillator.

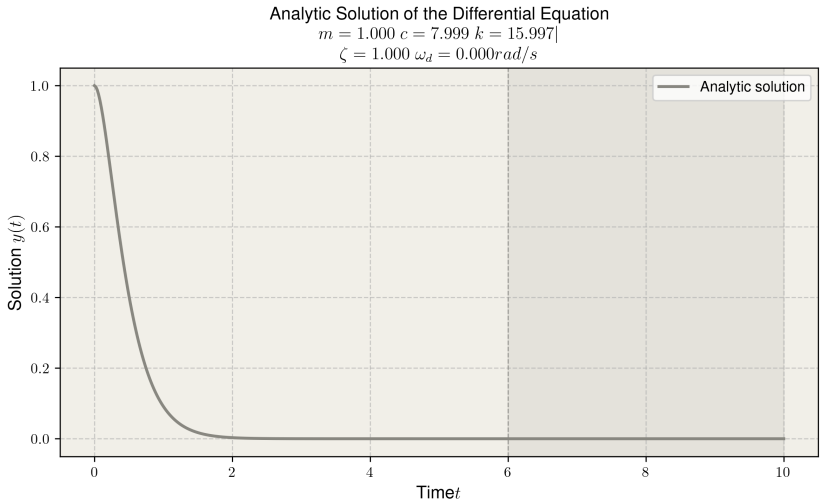


Figure 2.2: Analytic solution for the critically damped harmonic oscillator.

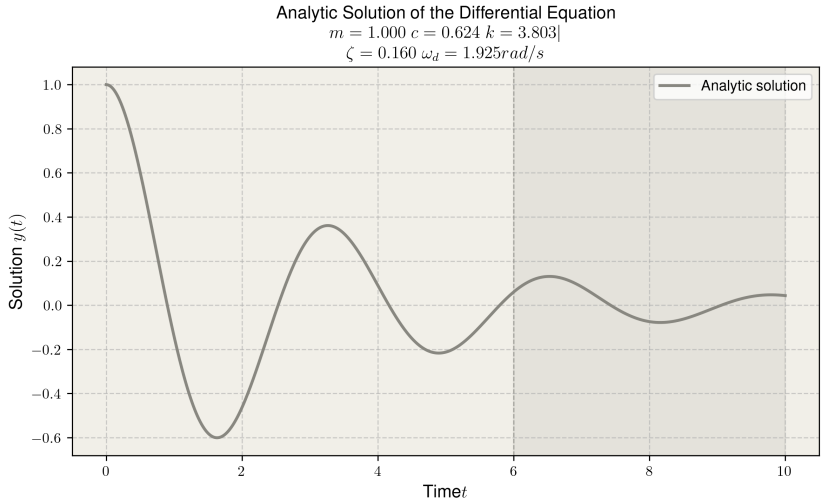


Figure 2.3: Analytic solution for the underdamped harmonic oscillator.

2.2 Understanding the PINN example

In this section, we will study the provided code `pinn_spring_damper.py` and attempt to respond to the questions about the method and the code. The provided example trains a standard ML model and a PINN to solve the underdamped case of the damped harmonic oscillator. The models in question are trained on 15 noisy observation points, which are supplemented with the initial conditions and the physics residual al 200 evenly spaced collocation points in the case for the PINN.

2.2.1 Questions on the method

Question 1

Identify the total loss function in the code and write its expression down in your report. Identify the different contributions to the loss function.

The total loss function has three terms and is displayed below:

$$\mathcal{L}_{tot} = \mathcal{L}_{data} + \lambda_{physics}\mathcal{L}_{physics} + \lambda_{i.c.}\mathcal{L}_{i.c.} \tag{2.11}$$

The first term is the mean squared error of the predicted data compared to the 15 noisy observation points:

$$\mathcal{L}_{data} = \frac{1}{M}\sum_{j=1}^{M}\left(\hat{y}(t_j) - y_j\right)^2 \tag{2.12}$$

with $j = 1, 2, \ldots 15$. This term is the total loss function of a standard, non-PI NN and gives the model direct information about the correct solution of the equation with the relevant boundary conditions, but doesn't check for any non-physical behavior.

The second term is the mean squared residual of the ODE (see equation 2.1) at the 200 evenly spaced collocation points:

$$\mathcal{L}_{physics} = \frac{1}{N}\sum_{i=1}^{N}\left(m\hat{y}''(t_i) + c\hat{y}'(t_i) + k\hat{y}(t_i)\right)^2 \tag{2.13}$$

with $i = 1, 2, \ldots 200$. This term gives the model information about how much the predicted solution violates the mechanics of the system, without checking if the predicted solution matches the observed data.

The third term is the sum of squared errors of the two inital conditions:

$$\mathcal{L}_{i.c.} = (\hat{y}(0) - y_0)^2 + \left(\hat{y}'(0) - dy_0\right)^2 \tag{2.14}$$

This term enforces the initial conditions and applies only to the very first point $t = 0$.

The constants $\lambda_{physics}$ and $\lambda_{i.c.}$ determine the relative weight of every term in the loss function. Because it is important that the initial conditions are met, $\lambda_{i.c.}$ is chosen at a much larger value than $\lambda_{physics}$. This is why in the given program, the values are set at $\lambda_{physics} = 0.01$ and $\lambda_{i.c.} = 10$.

Question 2

Is the inclusion of the initial condition loss essential for this particular example? When would it be essential?

To investigate the effect of the initial condition term, the term can be dropped out of the loss function to compare the results. Figure 2.4 shows the predicted solutions of the ODE if the $\mathcal{L}_{i.c.}$ is included and excluded. In this specific case, there does not seem to be a big difference which indicated that the initial condition loss is not essential. This is because the noisy observations point, which also contribute to the loss function, are close enough to $t = 0$ to give enough information for the model to accurately predict the initial conditions.

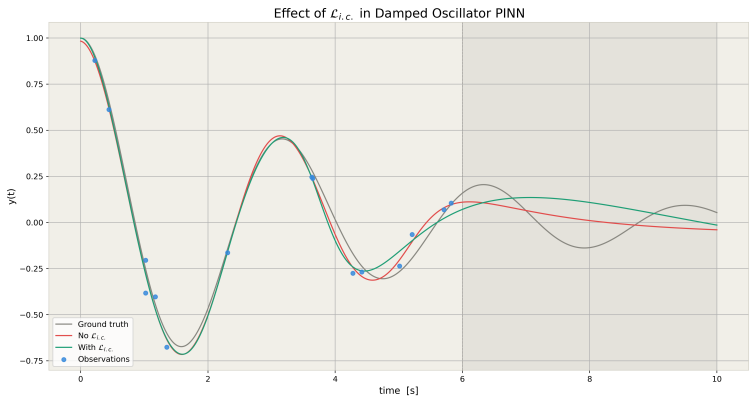


Figure 2.4: Predicted solutions to the ODE of the $\mathcal{L}_{i.c.}$ is included and excluded. In this specific case, the $\mathcal{L}_{i.c.}$ is not absolutely necessary, because the noisy observation points give enoug information for the model to accurately predict the initial conditions.

We do not always have access to observations close to the initial time t_0. However, if the initial conditions are known and need to be enforced, $\mathcal{L}_{i.c.}$ is a crucial term in the total loss function.

Question 3

The loss function used here is based on the mean squared errors (MSE) between predictions and data points or initial condition, and on the mean squared residual (MSR) of the PDE for the collocation points. Why are these specific functional forms used, and would there be alternative functions that could replace the MSE or MSR?

The mean square error and mean square residuals are the most commonly used loss functions for regression models. Any loss function has to be symmetrical and positive for all possible values, so that errors cannot cancel each other out. Furthermore, it is desired that the loss function is differentiable so that its gradient can be computed.

The MSE and MSR check both of these boxes. However, since the errors are squared before averaging, these functions heavily punish and are sensitive to large outliers. This can lead to problems when very noisy data points are included in the calculation for the loss function. However, this effect is good for the residuals since it is essential that the predicted solution does not break the laws of physics. There are alternative loss functions with different (dis)advantages [1]:

Mean Absolute Error The Mean Absolute Error (MAE) does not average the squares, but the absolute distances:

$$\mathcal{L}_{MAE} = \frac{1}{M}\sum_{j=1}^{M}\left|\hat{y}(t_j) - y_j\right| \tag{2.15}$$

Because the errors are not squared, this loss function is less sensitive to large outliers, which can make it more robust. However, it is not differentiable at zero, which can lead to problems for some optimisation algorithms.

Huber Loss The Huber Loss is a hybrid of the MAE and the MSE which is differentiable, but less sensitive to ouliers than the MSE. It is defined as follows:

$$L_\delta(x) = \begin{cases} \frac{1}{2}x^2 & \text{if } |x| \leq \delta \\ \delta\left(|x| - \frac{1}{2}\delta\right) & \text{if } |x| \geq \delta \end{cases} \tag{2.16}$$

The Huber Loss behaves like the MSE for $|x| < \delta$ and like the MAE for $|x| > \delta$. The hyperparameter δ can be tuned to make the loss function more or less sensitive to outliers.

Question 4

What is the type of neural network used in this example? How many input nodes, output nodes and hidden layers does it have? How many parameters are then being optimized? What type of activation function is used and why? What quantity or quantities are provided at the input, and what quantity or quantities do we extract from the output of the network? Are the dimensions of this Neural Network architecture adequate, too small or too large for this example? How would you determine this?

We are trying to solve an ODE, which means to solve for a function $y(t)$. This function takes one input and returns one output, which means the network will also need to have one input and one output, representing the time and the function value respectively.

A dense network is used in this specific case. The parameters for the hidden layers can be given as inputs, in the example script there are 4 hidden layers of 32 nodes. For every connection between nodes has a weight, and every node (except the input) has a bias. From this we can calculate the total parameter count:

$$\begin{aligned} n &= \text{First layer weights} + \text{First layer bias} \\ &\quad + \text{Hidden layer weights} + \text{Hidden layer bias} \\ &\quad + \text{Last layer weights} + \text{Last layer bias} \\ &= (1\cdot 32 + 32) + 3(32\cdot 32 + 32) + (1\cdot 32 + 1) = 3265 \text{ parameters} \end{aligned} \tag{2.17}$$

The dimensions of the network (the depth and layer size) are hyperparameters which should be tuned according to the specific problem needs to be solved by the model. Defining the optimal value for hyperparameters such as the dimensions is a relatively subjective and empirical process, since there are no general formulas that give the optimal hyperparameter set for every problem.

The activation function used is a tanh function. This choice is important; we will be calculating the first and second derivatives of the output y with respect to the input t, which means the activation function needs to be differentiable at least twice. for this reason, ex. ReLu is not suitable for this network.

Judging from the predicted solution, the model is large enough to make an accurate prediction for the solution to the equation. To optimise the network dimensions further, one can choose to implement an optimising algorithm which tunes the hyperparameters based on a validation set. This way, the acuraccy and training time can be optimised in a reproducible way [5]. Examples possible methods include:

- Random search: for every hyperparameter, a range of evenly spaced values is defined. A model is trained for every single possible combination and their performances are compared.
- Random search: hyperparameter sets are chosen at random with user-defined ranges, and the best performing model is chosen from the resulting set of models.
- Bayesian optimization: a probabilistic sequential algorithm that optimises an unknown, computationally heavy function (such as the model performance).

Question 5

How many input samples are provided to the neural network model during each of its training cycles or epochs? What kind of samples are provided? Does the network receive again for each epoch the same exact samples? Would there be a better way to provide samples to this network for training over the various epochs?

The model receives the same 15 noisy observation points every singe epoch. These points are used to calculate $\mathcal{L}_{data}$. Furthermore, the model receives the initial conditions to calculate $\mathcal{L}_{i.c.}$. The $\mathcal{L}_{physics}$ is calculated only using the residual of equation 2.1, for which no inputs are needed.

This is a suboptimal approach, since the model could simply memorize the 15 given points. A better approach would be to randomize the 15 points every epoch to avoid overtraining on a specific dataset. Furthermore, it would also be good practise to stop the inputting of datapoints after a defined number of epochs, so that the model is solely trained based on the equation residuals. This is even an even safer approach to avoid overtraining and is known as *curriculum training*. [6]

Question 6

The example uses the Adam algorithm as the so-called optimizer. What is the purpose of this optimizer? What is specific about the Adam algorithm, and what are its main hyper-parameters? Are there viable alternatives for the Adam optimizer?

An optimizer is a tool to accelerate the gradient descent by taking previously calculated gradients into account [2]. This way, the optimizer can avoid oscillations and avoid local minima. Adam is actually a combination of two alternative optimisers:

1. Momentum-based optimising [3]: this technische incorporates the gradients of previous epochs by calculating an exponentially weighted moving average. If the gradient at epoch t is given by w_t, the momentum-based gradient m_t is calculated iteratively as follows:

$$m_t = \beta m_{t-1} + (1-\beta)\nabla_{w_t}\mathcal{L}_{tot} \tag{2.18}$$

   where β is the momentum factor, typically chosen as 0.9. This parameter determines how much the previous gradients need to be taken into account. The weights are then updated:

$$w_t = w_{t-1} - \alpha m_t \tag{2.19}$$

2. Root Mean Square Propagation [4]: this technique uses an exponentially weighted moving average of squared gradients to optimise the learning rate α. The learning rate is divided by a factor $\sqrt{v_t}$, the root (weighted)mean square of the gradients. This is also calculated iteratively:

$$v_t = \gamma v_{t-1} + (1-\gamma)\left(\nabla_{w_t}\mathcal{L}_{tot}\right)^2 \tag{2.20}$$

The weights are then updated:

$$w_t = w_{t-1} - \frac{\alpha}{\sqrt{v_t}+\varepsilon}\nabla_{w_t}\mathcal{L}_{tot} \tag{2.21}$$

A small value is added in the denominator to avoid division by zero.

Both m and v are biased towards zero since they are initialized at zero. They need to be corrected:

$$\begin{aligned}\hat{m}_t &= \frac{m_t}{1-\beta^t} \\ \hat{v}_t &= \frac{v_t}{1-\gamma^t}\end{aligned} \tag{2.22}$$

The Adam optimiser combines these two techniques and updates the weights as follows:

$$w_t = w_{t-1} - \frac{\alpha}{\sqrt{\hat{v}_t}+\varepsilon}\hat{m}_t \tag{2.23}$$

The Adam optimiser is widely used since it generally has an efficient performance with relative little hyperparameters; only α, β, and γ need to be defined. Both the learning rate and the gradient itself are made adaptive which helps to find the global minimum of the loss function more effectively. Anternative optimisers inlcude the momentum-based and RMSprop techniques as discussed above, or Adamax (a variant of Adam based on the infinity norm) and Adagrad (which accumulated the gradients instead of using the weighted average).

Question 7

The example also uses an additional optimizer: a learning rate scheduler. What does this scheduler do? Is is a necessary ingredient for training a neural network? What are its main hyperparameters? Are there alternatives that can be usefully applied in this example?

The learning rate scheduler StepRL reduces the the learning rate with a factor γ after every n epochs, which are tunable hyperparameters. This adds a global decay, which can optimize the convergence in the different stages of optimisation. When the number of epochs is still small and the parameter set is still far from the optimum, the learning rate is big, but as the number of epochs gets bigger and the network converges towards the best set of parameters, the learning rate has to decrease as well in order to avoid oscillations around the minimum. This approach lead to a further acceleration of the gradient descent.

There exist many different alternative learning rate schedulers, such as:

1. ExponentialLR: similar to StepRL but apllies a small decay factor γ after every epoch, leading to an exponentially decreasing learning rate.

2. CosineAnnealingLR: the learning rate follows a cosine curve which converges to a minimum value.

3. ReduceLROnPlateau: adaptive scheduler which applies a decay factor γ when improvement stagnates.

4. CyclicalLR: non-traditional learning rate scheduler which periodically increases and decreases the learning rate in order to avoid local minima.

In this specific case, StepRL or ExponentialLR are good choices to effectively train the model.

Question 8

How many epochs is the network trained for? How would you determine whether you have trained for too little or too many epochs?

The model is trained for 8000 epochs. In order to determine if this is too many/too little, we need to compare the test and training loss function curves as a function of the number of epochs. When these two curves start to diverge, it means the model is overfitting on the training set. The model is then learning meaningless details in the test set. In the provided script no validation dataset is implemented, which makes it impossible to detect overfitting in this way. A proper implementation would reserve a separate set of points, unseen during training, against which the loss is evaluated each epoch.

Question 9

Would it make sense to have a set of validation points? How would these look like? How would you use them?

This script does not have a set of validation points, which makes it difficult to gauge the accuracy of the model while taking the possibility of overtraining into account. The most representative way to determine the accuracy of the solution is to compare it directly to the analytic, exact solution. This can be done by defining a set of many evenly spaced values fot t and calculating the analytic solution $y(t)$ at these points. This acts as the ground truth of the model. Then, the MSR between the predicted and ground truth datapoints can be calculated to give an idea of the accuracy of the model.

It could also be wise to split the validation points into two sets in order to more specifically invesitgate the models ability to extrapolate beyond the range in which observation points were supplied. The set would then be split up into a set of extrapolated and non-extrapolated points. Then, the mean squared errors of both sets can be compared to analyse how much worse the predictions for the extrapolated region are.

2.2.2 Questions on the code

Question 1

Why are these two lines of code good, or bad, for:

```
torch.manual_seed(42)
np.random.seed(42)
```

These lines of code set the seed for the random number generation in the script. Using the same seed every time makes it so that the noisy observation points are exactly the same when the script is ran. This can make it easier to debug the script when a specific combination of observation points causes a bug.

It is however also important to occasionally test the script with different seeds. The model architecture and the script could become optimised for this specific seed if other seeds are never tested.

Question 2

What is the purpose of the `np.ndarray` statements in this function definition:

```
def analytic(t: np.ndarray) -> np.ndarray:
```

This is Python's type annotation syntax, as prescribed by PEP484 [10]. It tells the user which datatypes the input and output need to be. No typechecking happens at runtime, so this statement doesn't explicitly raise a TypeError if an object of the wrong type is passed. It serves as dowumentation and can help the developer to keep track of the precise definitions of every variable.

Question 3

The data points are created randomly and then sorted before being offered to the neural net, using this statement:

```
t_obs = np.sort(np.random.uniform(0.1, T_TRAIN, N_OBS))
```

Is this good or bad?

Randomizing the datapoints is good practise. A periodic set of points could lead the model to memorize specific patterns, instead of predicting the general solution to the initial condition. However, as said before, this script always passes the same set of random points which negates the advantage of randomizing the points in the firs place.

The sorting of the points has no effect, since we are taking the MSE between the observation points and predicted points. The order in which the errors are summed has no effect on the loss function.

Question 4

The collocation points are created uniformly on a fixed grid, using this statement:

```
t_col = np.linspace(0.0, T_EXTRAP, N_COL)
```

Is this good or bad?

Using a uniform grid for the collocation points is not optimal, the model can fine-tune to the specific spacing of the collocation grid which is not desirable. It is better practise to generate the collocation points randomly at every epoch. By constantly regenerating a set of collocation points, any bias towards one region is also avoided.

Questions 5 and 6

What is this statement doing? Is it necessary?

```
optimiser.zero_grad()
```

What are these statements doing in the training cycle? Is their order important?

```
loss_total.backward()
optimiser.step()
scheduler.step()
```

This sequence of commands updates the model weights by calculating the gradients through back-propagation. `loss_total` is a 0-dimensional PyTorch tensor representing the loss function. The `loss_total.backward()` command will do a backward pass to see how the loss function changes with the model parameters (weights and biases), and apply the chain rule at every step in the calculation of `loss_total`. This results in a gradient which is *added* to the `.grad` attribute of every parameter. In other words, `loss_total.backward()` does not *save* the gradient, but *accumulates* the gradient. This means that the `.grad` attributes need to be set to zero before every backwards pass, which is exaclty what `optimiser.zero_grad()` accomplishes. This statement is absolutely necessary, without it the gradients would be accumulated in the `.grad` attributes which causes the weights and biases to be updated incorrectly.

`optimiser.step()` will read the `.grad` attributes and update the m_t and v_t values for every parameter and update the parameters accordingly (see eq. 2.23). m_t and v_t are not zeroed by the `optimiser.zero_grad()` command, because otherwise it would be impossible to implement the Adam optimiser since the gradients at previous epochs are needed.

`scheduler.step()` changes the learning rate according to the specific scheduler used. In this case, an internal counter of the number of epochs is kept, and once this counter reaches a multiple of 3000, the learning rate is halved.

The order of these commands are crucial. The gradients must be set to zero first to avoid accumulation of gradients. The gradients must then be calculated to avoid using the gradients of the previous epoch to update the parameters. Finally, the fresh gradients are used to update the parameters. [7]

Question 7

Why do the collocation time input tensor and the initial condition time input tensor require the gradient option, while the input data tensors do not require a gradient?

```
t_obs_t = to_tensor(t_obs)
y_obs_t = to_tensor(y_obs)
t_col_t = to_tensor(t_col, requires_grad=True)
t_ic_t  = to_tensor(t_ic,  requires_grad=True)
```

The collocation points are used to calculate the MSR of the ODE as shown in equation 2.13, and the initial conditions are used to calculate the deviation from the predicted initial conditions as shown in equation 2.14. In both expressions of the loss function, the derivatives of $y(t)$ show up, so these need to be calculated.

Rather than using finite differences to calculate the derivatives at the collocation points and at $t = 0$, we can calculate the gradients during the forward pass through the model using `torch.autograd.grad`. This is more efficient since it allows for the simultaneous prediction of $y(t)$ and the calculation of the derivatives. However, we need to tell the model which derivatives with respect to t are required, which is done by the `requires_grad=True` statement. For the calculation of the MSE of the noisy observation points as in equation 2.12, no derivatives are needed, so the statement is omitted and `requires_grad` is set to the standard value `False`. [7]

Question 8

When presenting arrays of time points to the input of my neural network, I need to convert them into a pytorch tensor, using this helper function:

```
def to_tensor(arr, requires_grad=False):
    return torch.tensor(arr, dtype=torch.float32,
                        requires_grad=requires_grad).unsqueeze(1)
```

What does the `unsqueeze(1)` method of the torch.tensor class do, and why do I need to do this?

The `unsqueeze()` method takes a pytorch tensor of rank n and adds a dimension of depth 1 at a specified index, as shown on Figure 2.5. In this case, the helper function takes in a one-dimensional array of length n and returns a $(n,1)$ `torch.tensor` class. This is necessary for two different reasons:

1. The time arrays (`t_obs_t` and `t_col_t`) which are used as inputs for forward passes through the network need to be $(n,1)$ shaped in order to do matrix multiplication. The first `nn.Linear(1, 32)` layer of the network performs the following matrix multiplication:

$$y = xW^T + b \tag{2.24}$$

   where x is the $n \times 1$ input of the first layer, consisting of n samples with 1 feature, which is the time value t. W and b are (32×1) column matrices containing the weights and biases[1] of the layer and y is the $n \times 32$ output. If we do not unsqueeze the time arrays, which correspond to x in this case, `nn.Linear(1, 32)` will interpret the $(n,)$ tensors as $(1,n)$ tensors, consisting of 1 sample with n features. This makes the matrix multiplication xW^T, which is $(1,n) \times (1,n)$ impossible. Note that the `t_ic_t` tensor does not need to be unsqueezed, since it is a $(1,)$ shaped tensor which cannot be interpreted incorrectly by the `nn.Linear(1, 32)` layer.

2. The `y_obs_t` unsqueezed tensor is not passed through the network, but rather is used to calculate the data loss $\mathcal{L}_{data}$ (see equation 2.12). When predicting the y values for the `t_obs_t` values, the model returns a $(n,1)$ tensor (`y_pred`). In the code, the residual is calculated as `l_data = torch.mean((y_pred - y_obs_t) ** 2)`. If `y_obs_t` is not unsqueezed ($(n,)$ tensor), the $(n,1) - (n,)$ operation will return a (n,n) tensor, containing *all* differences $\hat{y}_i - y_j$, instead of only the relevant errors $\hat{y}_i - y_i$. When averaging over these elements using `torch.mean`, it is averaging over differences and returns a loss which is not correct.

[1] b cannot actually be added directly to xW^T since the dimensions do not match. b is actually added to each row of the xW^T matrix.

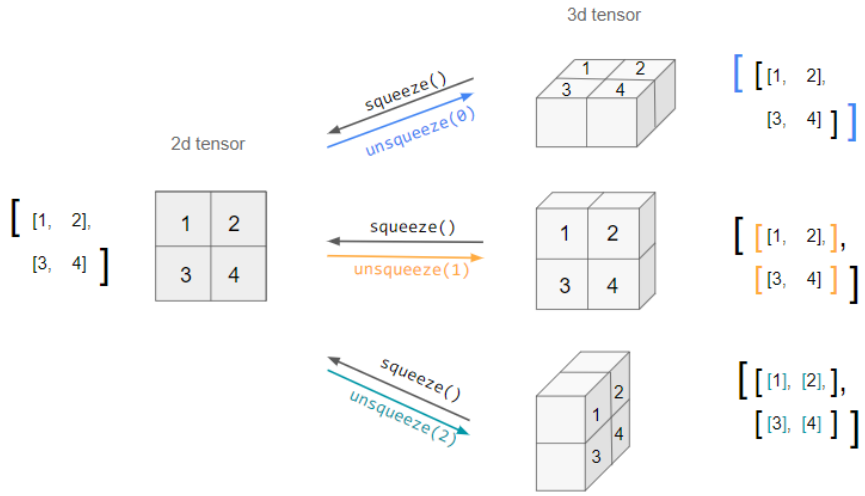


Figure 2.5: Unsqueezing of a 2×2 tensor into $1 \times 2 \times 2$, $2 \times 1 \times 2$, and $2 \times 2 \times 1$ tensors by using the `unsqueeze()` method of the `torch.tensor` class. [8]

Question 9

When computing the first and second derivatives of $y(t)$ we use the `torch.autograd` method. Explain why the `grad_outputs` are set to ones and why we need to take the zero-th element `[0]` of this output in the statements below:

```
dy = torch.autograd.grad(
    y_hat, t,
    grad_outputs=torch.ones_like(y_hat),
    create_graph=True)[0]
```

The `torch.autograd.grad` differentiates an output vector $y = (y_1, \ldots, y_n)$ with respect to an input vector $t = (t_1, \ldots, t_n)$ and returns a vector-Jacobian product:

$$v^T J = v^T \begin{pmatrix} \frac{\partial y_1}{\partial t_1} & \cdots & \frac{\partial y_1}{\partial t_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial t_1} & \cdots & \frac{\partial y_n}{\partial t_n} \end{pmatrix} \tag{2.25}$$

where v is set by `grad_outputs`. In our specific case, every value y_i only depends on the corresponding time value t_i, which makes the Jacobian a diagonal matrix. In order to contract the Jacobian to a column vector, v can be set to a row vector of ones with `grad_outputs=torch.ones_like(y_hat)`. The `torch.autograd.grad` acutally returns a tuple $(dy,)$ per input, so to unpack it we can simply select the first and only element using `[0]`.

2.3 Improving the example code

Now that we have a good understanding of the code and have a better idea of how to improve the code, we will restart from scratch and make a new implementatoin of the code based on the provided example. In this section, I will discuss the improvements I have made, demonstrate the code for the three different damping cases, and discuss the results and effects of different factors. Finally, we will adapt the code to solve the inverse problem, where I attempt to extract the physical parameters of the system without passing them as explicit inputs to the model.

2.3.1 Code improvements

To begin, I will give a thorough overview of the improvements I have made to the code. These rane from fundamental changes to the training process and code structure, to more minor changes to the plotting methods and helper functions.

2.3.1.1 File structure

While the original code was implemented in a single script, I have decided to split the different functionalities and steps into multiple files. The file structure is as follows:

Damped harmonic oscillator file structure

```
Damped_Oscillator/
|-- config.py           - dataclass holding all physical constants,
|                        hyperparameters, runtime settings
|-- data.py             - analytic solution and data generation for
|                         collocation points, observation points,
|                         and initial conditions
|-- model.py            - FCNet architecture and predictor functions
|-- losses.py           - loss functions for the training of the model
|-- trainer.py          - training loop
|-- plot.py             - all plotting functions
|-- utils.py            - helper file for small functions
```

This file structure allows for a more modular code, which makes it more accessible and easier to maintain or debug. This also allows me to make multiple scripts for different experiments easily, since the necesarry functionalities can simply be imported from the relevant files. The `config.py` file is especially useful, since it provides a dataclass thatallows for the user to specify all parameters at once. This class can then be passed to the different functions and classes which automatically sets the correct parameters. Splitting the trainer script and the model architecture is also a flexible approach, since it allows for modular implementation of new models (ex. a model for the inverse problem, see section 2.4) or new training strategies (ex. curriculum training).

In order to train a model using these files, a separate script needs to be created. The workflow is as follows:

1. We check if there is a GPU available and set the device accordingly using the `get_device` helper from the `utils.py` file.

2. A `Config` object with user-defined parameters is created.

3. Data is generated based on the ground truth solution through the `generate_data` function from the `data.py` file. This includes the collocation points, observation points, and initial conditions.

4. A `FCNet` model, from the `model.py` file, is created based on the parameters in the `Config` object.

5. The model is trained using the `train` function from the `trainer.py` file, which takes the model, data and `Config` object as inputs.

In the rest of this report, we will design multiple of scripts like this to do different experiments. The code for these scripts can be found along with the rest of the aforementioned files in the `Damped_Oscillator` directory.

2.3.1.2 Randomising training data

In the provided example, the model is trained on a training set of 15 noisy observation points. The same training points are used at every single epoch, which could lead to the model overfitting or bias towards one region of the time domain as explained previously. To avoid this, I have chosen to randomise the observation points at every epoch. This gives the model more spread-out information about the entire time domain of the equation. I have chosen to keep these observation points noisy for two reasons:

1. Noisy observation points is more representative of real-world data, which is never perfectly clean. We are essentialy producing 'fake' observation data fot the model to train on, which is a technique that is widely used in the field of machine learning when the amount of real data is limited.

2. The noise can also be seen as a way of forcing the model to learn the physics of the system, rather than only memorising the numerical value of the provided observation points. This can avoid overfitting and lead to better results when extrapolating the model beyond the training domain.

Just like the observation points, the collocation points are also randomised. In the provided example the same dense evenly spaced grid of collocation points is used, in my code the set of collocation points is randomly chosen at every epoch. Since there are way more collocation points than observation points, this is more to avoid the model to fine-tune to the specific spacing of the collocation grid, which is not desirable.

2.3.1.3 Early stopping

In the provided example, the model is trained for a pre-set number of epochs. This is not optimal, since it could be possible that the model has already converged to an optimal set of parameters before all epochs have passed. to account for this, we need to implement an early stopping strategy which stops the training when the model has converged. For this, we need to choose a performance metric to monitor during the training, so that the training will stop when the metric has minimalised and the improvement has stalled.

The most representative way to determine the accuracy of the model is to compare the predicted solution to the exact analytic solution. The metric which we want to use is the mean squared error between the predicted and exact solution over the interval:

$$MSE = \int_0^{t_{end}} (y(t) - \hat{y}(t))^2 dt \tag{2.26}$$

In our case, this integral can be approximated by choosing a dense, evenly spaced grid and approximating the integral by the Riemann-sum over the interval:

$$MSE = \sum_{i=0}^{N} (y_i - \hat{y}(t_i))^2 \Delta t \tag{2.27}$$

This metric will be called the *validation loss*, annotated by $\mathcal{L}_{val.}$. The training will stop when the validation loss has stopped decreasing. We need to define two more parameters to get an efficient early stopping strategy:

1. The patience is defined as the number of epochs to wait after the last improvement (decrease of the validation loss) before stopping the training. This is necesarry to avoid stopping too early, since it is possible that the validation fluctuates around (local) minima and further improvements can be made after more training epochs. If the patience is ran out, it means the model has not improved for a while and training can be safely stopped.

2. The patience threshold is defined as the minimum improvement in the validation loss to be considered a *significant* improvement. only if the improvement is large enough and significant, the patience counter will be reset. This avoids resetting the patience counter for minimal, unsignificant improvements and makes the early stopping strategy more efficient.

Both of these parameters can be set in the `Config` class.

2.3.1.4 Model saving

Another improvement is the saving of the best model weights after every training run. This allows us to load the trained model later instead of retraining the model every time we want to make or show a predicition. Throughout the training loop, the weights that give the lowest validation loss are saved as a variable `best_state`. After the training has completed, these weights are passed as an output of the function along with the history of the different loss terms as a function of the epochs, and the prediction snapshots after a pre-set number of epochs. I have then created a helper funtion called `save_model` that saves these the relevant outputs to a folder. The model weights and snapshotsare saved as `.pt` files, the history is saved as a `.csv` file, and the `Config` parameters are saved as a `.json` file.

A second helper function, `load_model`, is implemented to reload the best model weights from a specific training run and do predictions using those weights. This is done by creating a new `FCNet` using the saved `Config` parameters, afetr which the best weights are loaded into the model using the `load_state_dict` method. The loaded model is then returned along with the training history, snapshots and `Config` parameters. This model is then ready to be used to make predictions and plot results.

2.3.1.5 Plotting improvements

The plotting methods from the provided example are more or less reused with some small improvements. The following plotting methods are implemented in the `plot.py` file:

- A plot of the analytic ground truth solution of the differential equation (`plot_analytic`). This function is entirely implemented by me.
- A plot of the predicted solution using the best weights of the model, along with the analytic ground truth solution (`plot_predicted`). This function is entirely implemented by me.
- A plot of the saved snapshots, showing the predicted solution at different stages of the training process (`plot_epoch_figure`). This function is taken from the provided example.
- A plot of the different loss terms as a function of the epochs (`plot_loss_history`). This function is entirely implemented by me.
- A summary plot with six panels (`plot_summary`). This plot shows:
    1. The analytic solution with noisy observation points and the position of the collocation points[2].
    2. The analytic and predicted solutions on the training interval.
    3. The training loss $\mathcal{L}_{tot}$ (logarithmic scale) as a function of the epochs.
    4. The analytic and predicted solutions on the training and extrapolated interval.
    5. The pointwise data residual, in other words the squared error between the predicted and observed values $(y_i - \hat{y}(t_i))^2$, as a function of time.
    6. The squared ODE residual $|r(t)|^2$ as a function of time.

[2]These points are actually never usen in training, but are generated after training has already finished. They do however give an intuïtive idea of what the training set look like, which is why I have chosen to plot these as well.

Panels 1, 2, 3, and 4 are copied from the provided example, while panels 5 and 6 are made by me (based on the original fifth panel, which showed the *absolute* error between the predicted and observed values).

These plotting methods are then wrapped in a single function called `save_model_plots`, which generated all of the above plots and saves them to a specified output folder. This function can be called at the end of a training loop witht he model, training history, epoch snapshots, data, and `Config` parameters as inputs.

Alternatively, one can choose to save the outputs of the training loop to a folder and then call the `save_plots_from_file` funtion, which load the saved model weights and saves all relevant plot to the same folder. This is my preferred approach, since the best model weights need to be saved in this way, which can be useful in the futire for ex. comparing model performances.

Examples of all plotting methods are shown in section 2.3.2.3

2.3.2 Code demonstration

In this section, I will give a quick demonstration of three PINNs, used to predict the solution to each of the three damped cases. In all three examples, we will use the following hyperparameters:

Table 2.1: PINN Hyperparameters

Parameter	Description	Value
`hidden`	Hidden units per layer	64
`n_layers`	Number of hidden layers	5
`n_epochs`	Maximum training epochs	30000
`lr`	Initial learning rate	5×10^{-3}
`patience`	Early stopping patience	1000
`patience_threshold`	Min improvement to reset patience	1×10^{-5}
`lambda_phys`	Physics loss weight	1×10^{-2}
`lambda_ic`	Initial condition loss weight	1×10^{1}

For now, these hyperparameters are chosen because they are observed to give good results. Section 2.3.5 will give a more quantitative analysis of the effect of the hyperparameters. Furthermore, the collocations points are chosen in the training *and* extrapolated domain. In this way, we are not actually making a fully blind prediction about the extrapolated domain, rather a physics informed prediction. This distinction will be further analysed in section 2.3.3. For brevity, I will only show the summary plots. All generated plots can be found in the `outputs` directory. Below, I will discuss each case separately.

2.3.2.1 Overdamped demonstration

For the overdamped case, I chose the following parameters:

Table 2.2: Overdamped parameters

Parameter	Value
ζ	1.5
ω_0	2

The training was stopped early after $\sim$ 1600 epochs and the final prediction yields $RMSE = 0.00485$. The results of the training are shown on Figure 2.6.

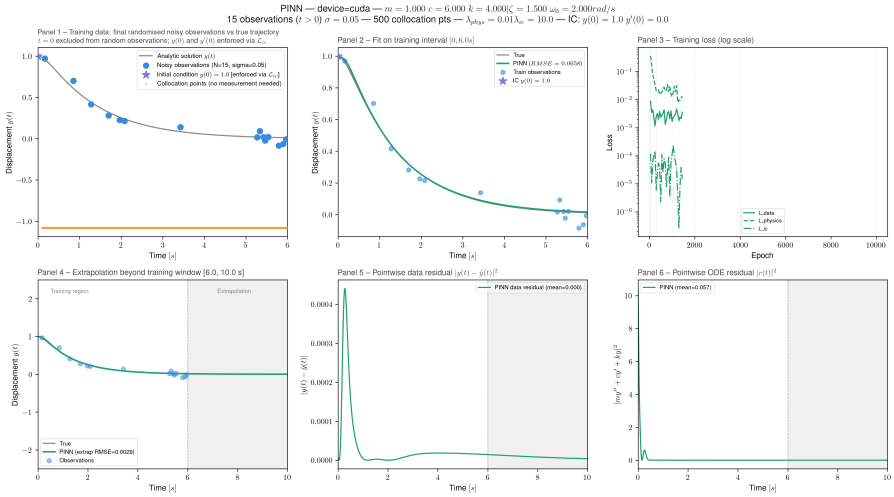


Figure 2.6: Summary plot for the overdamped case.

We can see that the model can predict the solution of the ODE very accurately after a relatively small number of epochs. As a reminder, the provided code ran the training for a set number of 8000 epochs. However, the overdamped case is a relative simple solution, so it is not surprising that the model is able to predict the solution.

When looking at the MSE and physics residual plots, we can see that the model struggles the most with the inital part of the solution. One possible explanation is that the $\lambda_{i.c.}$ is so large that it essentialy fixes the starting point and derivative in place, at the cost of the other terms in the total loss function. Furthermore, the observation points are actually generated from $t = 0.1$ to $t = t_{dom}$ to avoid accounting for the initial condition twice, so it is not all that strange that the model has the highest MSE for $t < t_{dom}$

We can see that the model perfroms exceptionally well for the extrapolated domain. However, since the solution to the ODE is virtually zero, this is not an indicator for the model performance. The extrapolating capabilities will be interpreted when the solution is not stationary in the extrapolated domain, such as for the underdamped case.

2.3.2.2 Critically damped demonstration

For the critically damped case, I chose the following parameters:

Table 2.3: Overdamped parameters

Parameter	Value
ζ	1
ω_0	2

The training was stopped early after $\sim$ 1400 epochs and the final prediction yields $RMSE = 0.00480$. The results of the training are shown on Figure 2.7.

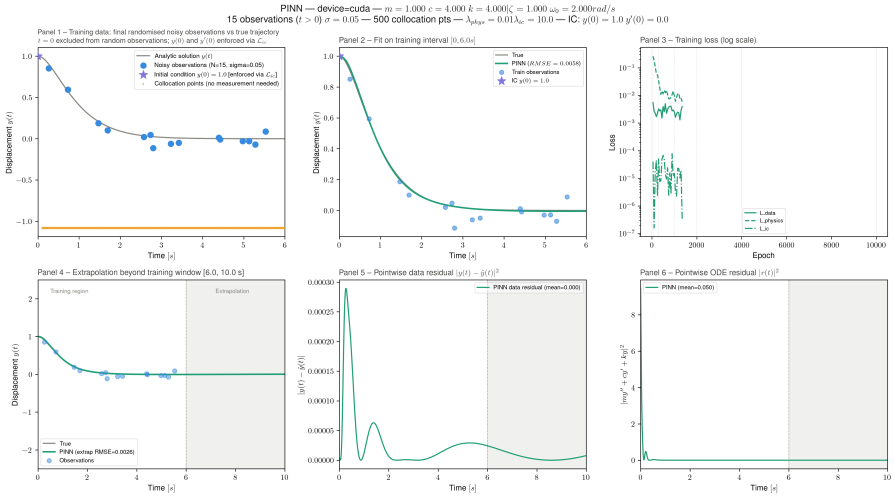


Figure 2.7: Summary plot for the critically damped case.

The critically damped case is very similar to the overdamped case, as it can be seen as a limit case of the overdamped solution. The *RMSE* and number of epochs to reach a solution are very similar indeed. The validation MSE and the physics residual plots also look very simular to those shown on Figure 2.6. Again, the extrapolation is very good, but trivial at $y = 0$.

2.3.2.3 Underdamped demonstration

For the overdamped case, I chose the following parameters:

Table 2.4: Overdamped parameters

Parameter	Value
ζ	0.125
ω_0	2

The training was stopped early after ~ 4700 epochs and the final prediction yields $RMSE = 0.00562$. The results of the training are shown on Figure 2.8.

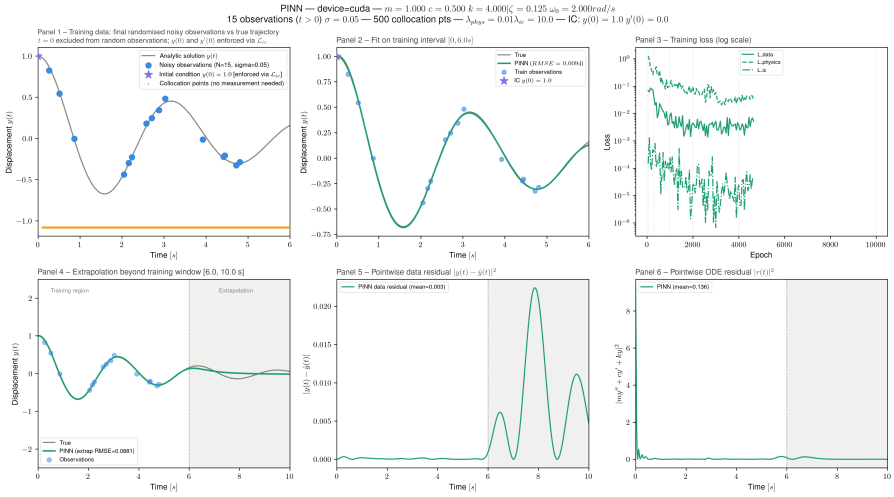


Figure 2.8: Summary plot for the underdamped case.

The underdamped case is the most interesting case of the three, and also the case in the provided code. First of all, the number of epochs until the optimisation stalls is a factor ~ 3 greater than the other two cases. This makes sense as the oscillatory behavior of this solution is more complex than the converging behavior of the other cases, which takes more time for the model to understand.

Here, the extrapolating capabilities become clearer. The model is not capable of extrapolating the exponentially suppressed oscillaion, but flattens out to a straight line at $y(t) = 0$. This is the most trivial solution of the ODE, and the actual analytic solution for $t \to \infty$, so it is not surprising that the predicted solution converges to zero in the extrapolated domain. It seems that the model has not yet learned to continue the oscillation in the extrapolated domain. These results are however very promising, and can be improved with proper hyperparameter tuning (see section 2.3.5).

Since the underdamped regime is the most interesting, I will show the rest of the generated plots for this case as well as a demonstration of all plotting methods. The plotting methods are shown on Figures 2.9 to 2.12.

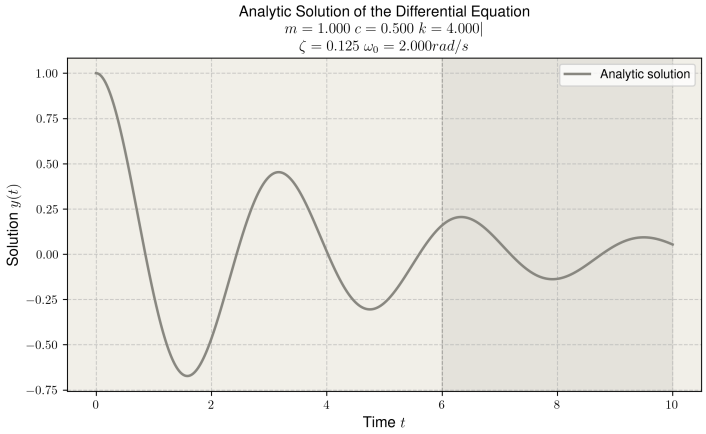


Figure 2.9: Predicted solution for the underdamped case.

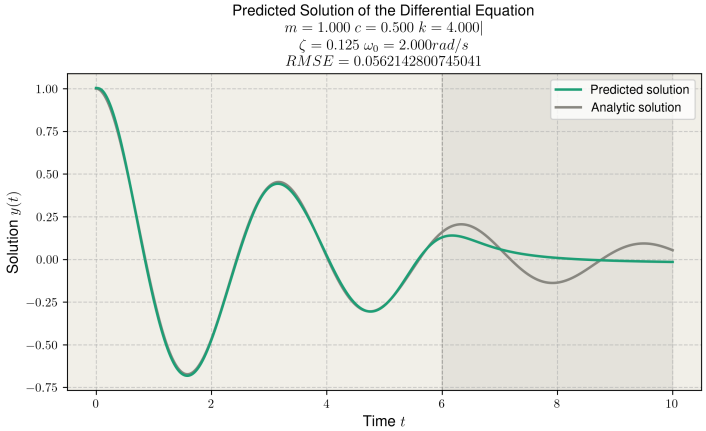


Figure 2.10: Predicted solution for the underdamped case.

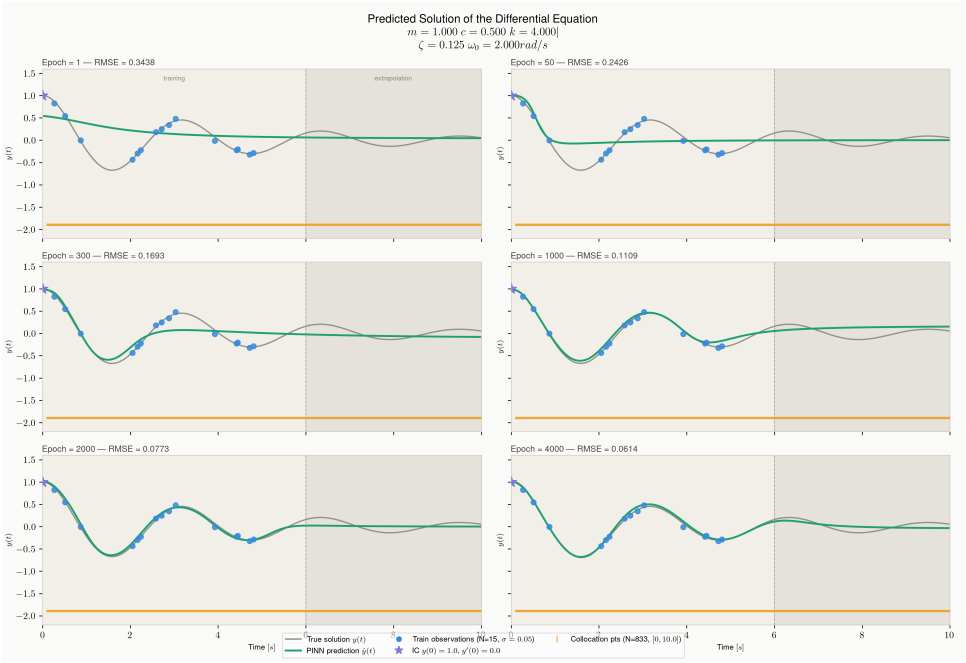


Figure 2.11: Epoch snapshots for the underdamped case.

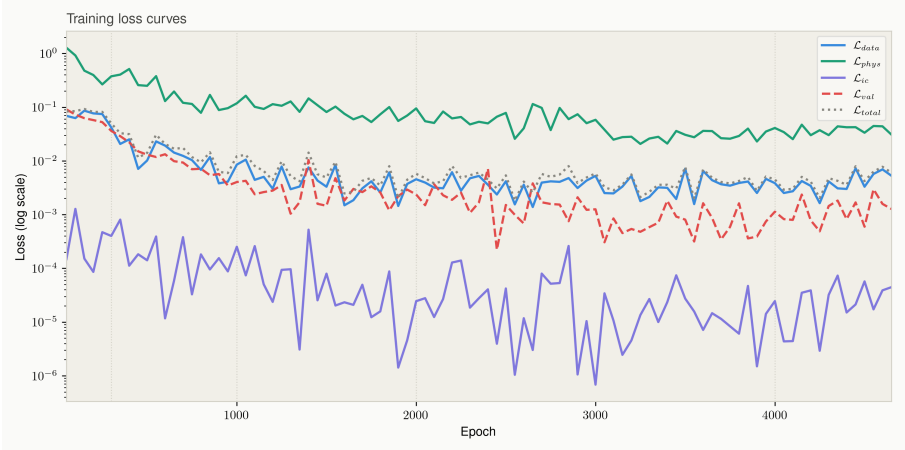


Figure 2.12: Loss convergence plot for the underdamped case.

These plots give us some useful insights. The epoch plot (Figure 2.11) shows how the model learns the equation behavior, starting at low t and working its way through the entire domain. The loss plot (Figure 2.12) is also very interesting. We can see that the $\mathcal{L}_{i.c.}$ is minimised the fastest, as it is the largest contribution to the total loss function. At that point, the largest contribution to the total is form the data, and the $\mathcal{L}_{data}$ and $\mathcal{L}_{total}$ curves match each other quite well. In this case $\mathcal{L}_{val}$ stays lower than $\mathcal{L}_{data}$, since the observation points have data applied to them but the validation points do not. Over time, the other loss terms also converge, and at some point the validation loss $\mathcal{L}_{val}$ stops decreasing; this is the point where the model starts overfitting. Since no more significant optimisations are made, the training is stopped.

2.3.3 Comparison between blind and physics informed extrapolation

Before tuning the hyperparameters or attempting to solve the inverse problem, I want to study the effect of some of the loss terms. Firstly, in all previous simulations, the collocation points were allowed to be chosen in the extrapolated domain. This makes the extrapolation not fully blind, but physics-informed.

In this experiment, the collocation points will only be chosen in the training domain, and no other hyperparameters will be changed.. This results in an actually blind test, as shown in Fugure 2.13

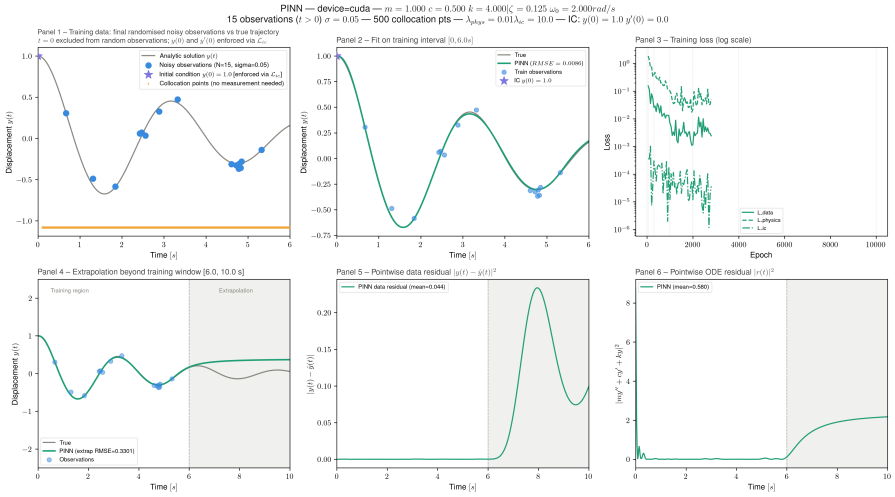


Figure 2.13: Summary plot for a PINN whithout collocation points in the extrapolated domain. This leads to a fully blind predicition for $t > t_{dom}$.

In the training domain, the predicted solution is virtually the same as in the previous example (Figure 2.8). In the extrapolated domain however, the results are different. the solution also plateaus, but now stays constant at around $y = 0.4$ instead of $y = 0$. This is unphysical, but since the physics loss is not calculated in the extrapolated domain, the model is not punished for this. The model has not generalised, and since there are no loss terms dependent on the predicted solution in the extrapolated area, the predicted solution simply remains constant.

2.3.4 Comparison with traditional ML model

Another interesting experiment is to compare the results to a non-physics informed neural network, as was done in the provided code. For this, the physics terms $\mathcal{L}_{phys}$ and $\mathcal{L}_{i.c.}$ are dropped from the loss function, and the model is retrained with the same hyperparameters. This gives the following results:

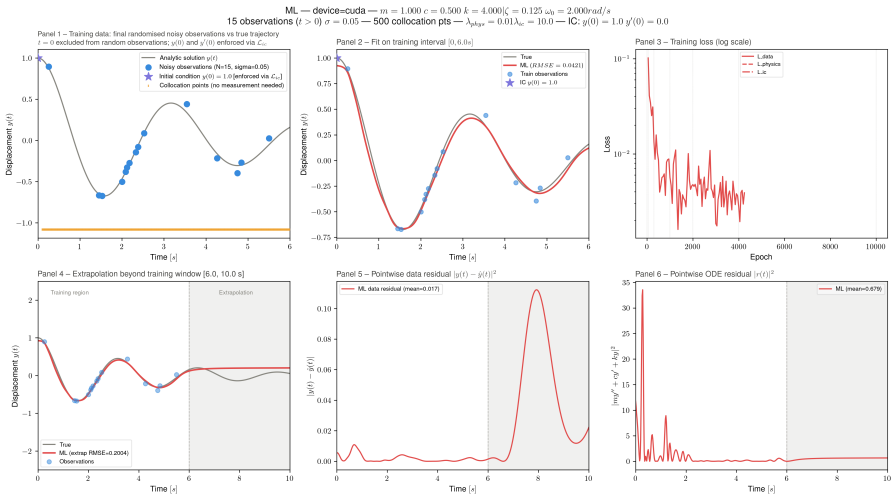


Figure 2.14: Summary plot for a non-physics informed neural network.

Visually, the solution does not look all to different from the PINN prediction. However, when looking

at the physics residual plot, we can see that $r(t)$ is very large when compared to the PINN. This is precisely the strength of PINNs; while standard neural networks can fit to the data and interpolate between observation points, they can overfit and/or lead to unphysical predictions, while a PINN both fits the data and complies to the ODE.

In the extrapolated domain, the results are very similar to the blind test PINN extrapolation. This makes sense, both the standard neural network and the blind test PINN do not have any information about the extrapolated domain, and we have already established that the PINN cannot generalise if it does not informed about the physics in the extrapolated domain. This makes it so thet predict more or less the same behavior in the extrapolated domain.

2.3.5 Hyperparameter optimisation

Instead of choosing the hyperparameters through trail-and-error, it is good practise to do a structured hyperparameter optimisation to understand the effect of the most important parameters and find the optimal configuration for eficcient model training. In this section, we will investigate the effect of some of the hyperparameters, and then use the `optuna` library to employ a *Tree-Structured Parzen Estimator*, which can tune the hyperparameters in an automised way. Bayesian optimisation is a widely used technique to efficiently find an optimised set of hyperparameters [11].

I have chosen to focus on the underdamped case, as it has the most interesting dynamics of the three cases. The bayesian optimisation will howveer be performed on all three cases. As a reminder, Figure 2.10 shows the predicted solution using the hyperparameters shown in Table 2.1.

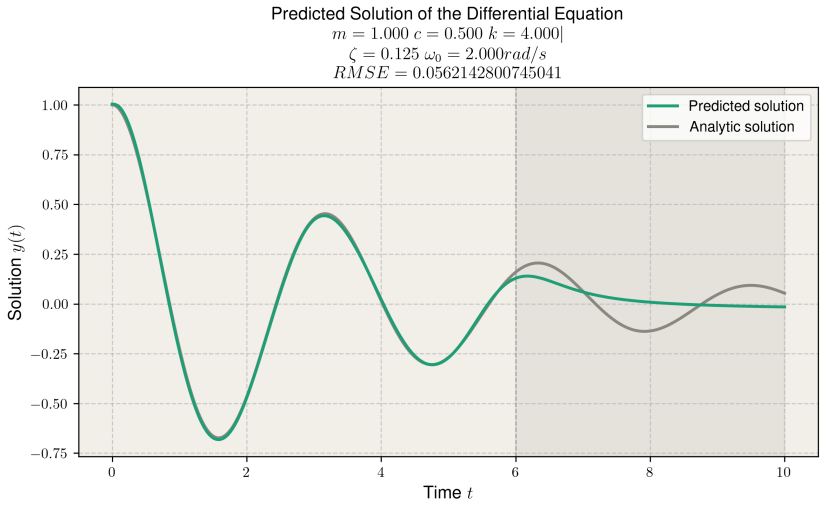


Figure 2.15: Predicted solution with standard hyperparameters shown in Table 2.1

2.3.5.1 Model complexity

The first two parameters we will investigate determine the *model complexity*, in other words the number of parameters to be optimised. To investiugate this, we will train a model with a very low complexity, more specifically `n_layers` $= 2$ and `hidden` $= 4$. This model thus only has 8 nodes, divided between two layers. This ersults in the following fit:

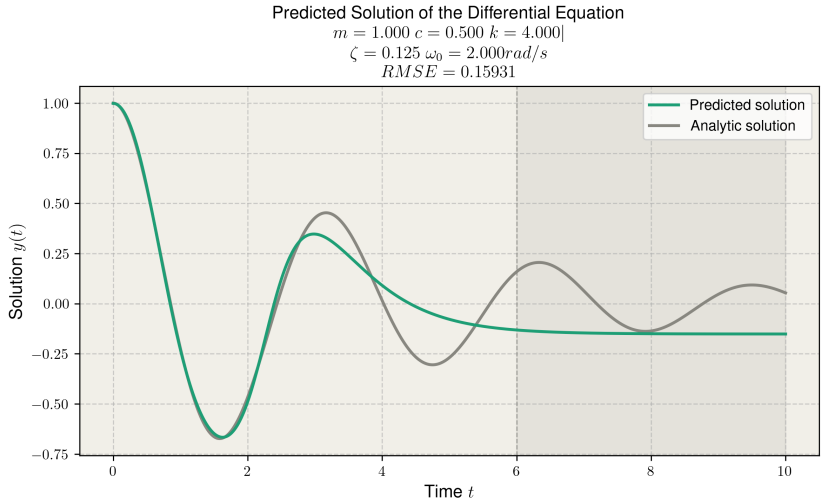


Figure 2.16: Predicted solution from a model with a very low complexity.

This model is not able to fit the analytic solution over the entire training domain. The complexity is too low for the model to fit to multiple oscillations, so the improvements stall after the prediction has fitted to the first oscillation of the analytic solution. As explained before, it is important to tune the model complexity s that the model is complex enough for the given task, but not overly complex to avoid overfitting or conputationally model heavy training.

2.3.5.2 Learning rate

Another cricual parameter is the learning rate of the model. I will study two cases; one where the learning rate is chosen too high and one where the learning rate is chosen too low. When the learning rate is set very high, say $\texttt{lr} = 0.05$, we get the following results:

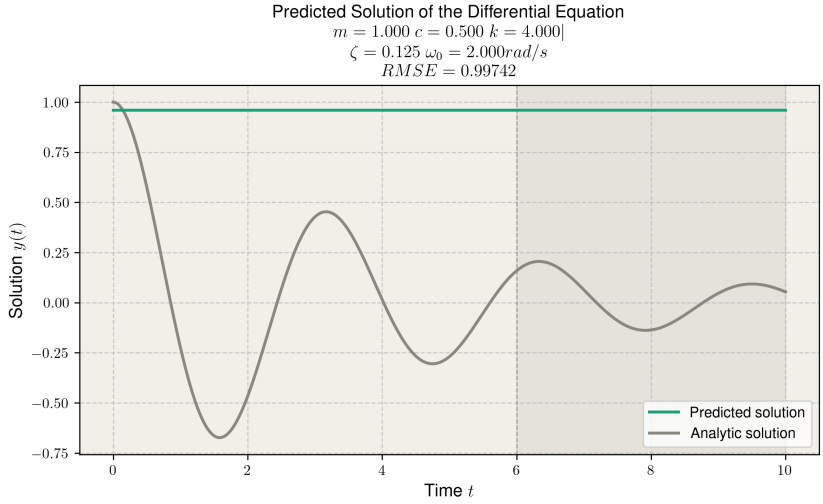


Figure 2.17: Predicted solution with $lr = 0.05$.

The learning rate is chosen so high that the model does not converge to an optimal solution at all. Rather, $\hat{y}(t)$ remains constant over the entire interval. When running this training, the early stopping mechanism triggered after 1014 epochs, which is a little over the patience limit. This indicated that the model was never really optimising.

When the learning rate is set very high, say $\texttt{lr} = 0.001$, we get the following results:

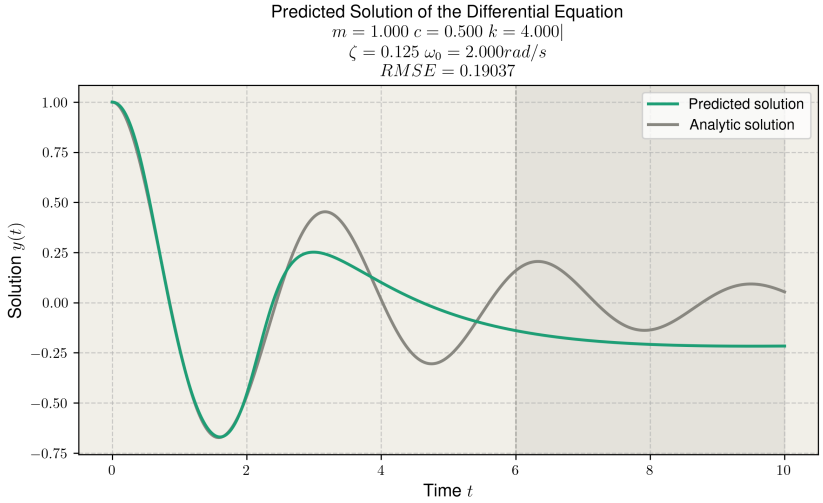


Figure 2.18: Predicted solution with $lr = 0.05$.

The model has not fully fitted to the analytic solution. This is the case because the early stopping mechanism has triggered too early; the small learning rate leads to very gradual improvements, which leads to the patience limit being reached since no significant improvement has been made. From these results, it can be conlcuded that the learning rate has to be tuned very precisely to get an efficient training loop.

2.3.5.3 Loss function weights

We have already investigated what happend when λ_{phys} is set to zero. Let us now choose a very high value, say $\lambda_{phys} = 100$. This results in the following solution:

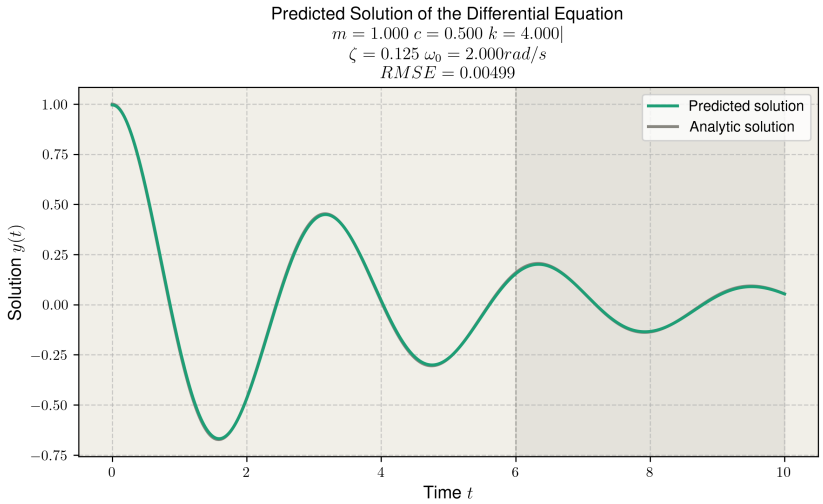


Figure 2.19: Predicted solution with $\lambda_{phys} = 100$.

Surprisingly, the model can suddenly pake an almost perfect prediction about the extrapolated domain. The large contribution of the physics term in the loss function seems to have forced the model to learn the solution of the ODE even without observation points. This reposes the question if a fully blind

generalisation is possible. If we do not privide collocation points in the extrapolated domain, but keep $\lambda_{phys} = 100$, we get:

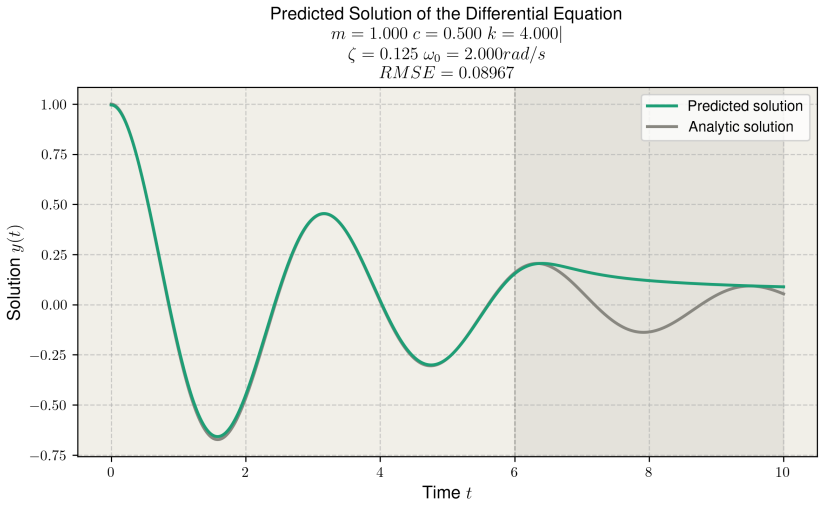


Figure 2.20: Predicted solution with $\lambda_{phys} = 100$ and no collocation points in the extrapolated domain.

The model does not seem to dit to the oscillatory behavior, but it does decrease exponentially, which means it at least has some notions of what the solution would look like. This was not the case in the previous blind test, where the solution evened out at a constant value (see Figure 2.13).

Finally, I want to see if the model can fit to the solution **??** any observation points. This results in the following prediction:

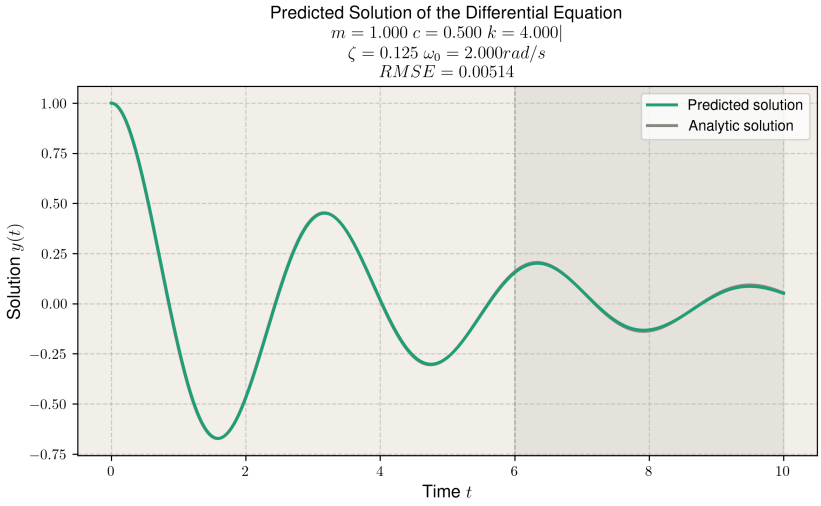


Figure 2.21: Predicted solution with $\lambda_{phys} = 100$ and no noisy observation points for training.

The model is in fact able to reconstruct the analytic solution without any data points being provided. This does however negat the point of using a neural network, since it is also possible to reconstruct the solution with finite-difference methods (ex. Euler method or Runge-Kutta). PINNs are more applicable to backwards problems, where we need to extract the (unknown) physical parameters from observational data. This means that we need to balance the loss terms so that both the observation points and the physics of the system are taken into account in the training loop.

2.3.5.4 Optuna

In order to find an optimal model complexity and learning rate, as well as tune the balance of the loss function, I have chosen to use the implement a use the `optuna` library to employ a *Tree-Structured Parzen Estimator* with the `optuna.samplers.TPESampler` class. This has a few advantages [11]:

- The optuna library can *prune* runs that do not look promising enough to provide an optimisation of the hyperparameters. This makes the optimisation more efficient since not every run needs to be fully completed.
- Bayesian optimisation is an efficient technique of finding the minimum of black box functions. In this case the black box function is the *MSE* of the best trained model.
- Tree-Structured Parzen Estimators are a subclass of Bayesian Optimisation algorithms which are efficient for high-dimensional spaces, which is applicable here since we have to tune many hyperparameters in order to get an optimal configuration.

We have chosen to separate the three cases, since the dynamics are very different for over- and underdamped cases which means the optimal hyperparameters will likely also be different. We have chosen to tune the following parameters in the following intervals:

Table 2.5: Optuna Hyperparameter Search Ranges

Parameter	**Description**	**Search Range**
`hidden`	Hidden units per layer	$[16,\ 128]$, step 16
`n_layers`	Number of hidden layers	$[2,\ 6]$, step 1
`lambda_phys`	Physics loss weight	$[10^{-3},\ 10^{1}]$ (log)
`lambda_ic`	Initial condition loss weight	$[10^{0},\ 10^{2}]$ (log)
`lr`	Initial learning rate	$[10^{-4},\ 10^{-2}]$ (log)
`scheduler_gamma`	LR scheduler decay factor	$[0.3,\ 0.9]$
`scheduler_step`	LR scheduler step size	$[1000,\ 5000]$, step 1000

For each case, we will run 1000 trials and finally save the configuration that leads to the lowest MSE to a `.json` file. This yielded the following results:

Table 2.6: Optimal Hyperparameters Found by Optuna

Parameter	**Description**	**Overdamped**	**Critically Damped**	**Underdamped**
`hidden`	Hidden units per layer	96	128	64
`n_layers`	Number of hidden layers	5	4	4
`lambda_phys`	Physics loss weight	0.78	3.78	0.27
`lambda_ic`	Initial condition loss weight	1.45	5.51	2.07
`lr`	Initial learning rate	9.90×10^{-4}	7.64×10^{-3}	3.65×10^{-3}
`scheduler_gamma`	Scheduler decay factor	0.553	0.545	0.548
`scheduler_step`	Scheduler step size	4000	2000	4000

The optimal configurations are relatively similar. The optimal model complexity has $\sim 10^4$ parameters, the physics loss weight is $\sim 10^0$, the learning rate is $\sim 10^-3$, the scheduler decay factor is ~ 0.5 and the scheduler step size is ~ 3000. We can see that the optimiser has not maximalised the physics loss weight, which indicated that the inclusion of noisy observation data in the training loop is still a good choice. Based on these results, we can make well-reasoned choices for the hyperparameters in the inverse problem.

2.4 Inverse problem

We will now try to extract the parameters ζ and ω_0 from the equation by training the model on noisy observation data generated from an analytical solution with unknown, randomised parameters. This can be implemented easily by creating a new model achitecture class, `InverseFCNet` with $\hat{\zeta}$ and $\hat{\omega}_0$ as model parameters and a new loss function `loss_physics_inverse` which uses these parameters. $\hat{\zeta}$ and $\hat{\omega}_0$ are then treated like every other weight or bias parameter, and will be tweaked every epoch until the model training is complete.

Based on the previous optimised hyperparameters, I have chosen for the following hyperparameters for the inverse problem:

Table 2.7: Hyperparameters used for inverse problem

Parameter	Description	Value
`hidden`	Hidden units per layer	96
`n_layers`	Number of hidden layers	5
`lambda_phys`	Physics loss weight	0.5
`lambda_ic`	Initial condition loss weight	10
`lr`	Initial learning rate	2.0×10^{-3}
`lr`	Initial learning rate for $\hat{\zeta}$ and $\hat{\omega}_0$	1.5×10^{-2}
`scheduler_gamma`	Scheduler decay factor	0.8
`scheduler_step`	Scheduler step size	3000

I have chosen to slightly increase the λ_{phys} than the optimal configurations found by optuna. By increasing the physics loss contribution, I am forcing the model to prioritize minimising the ODE residual, for which good estimates for the parameter are necesarry. Hovewer, I want λ_{ic} to remain the largest loss term to fix the initial condition in place[3], so this term as increased as well.

In this section, I will discuss three demonstration examples where the paramaters for an overdamped, critically damped, and underdamped case are extracted. Then, I will provide a statistical analysis where the parameters are estimated for a great number of randomised ground truth solutions.

2.4.1 Overdamped case

For the overdamped demonstration example, the randomised and predicted parameters are:

Table 2.8: Overdamped parameters

Parameter	True value	Predicted value
ζ	1.162	1.556
ω_0	$2.523 rad/s$	$2.592 rad/s$

Figures 2.22 and 2.23 show the predicted solution and parameter convergence to the true values.

[3]This is actually not absolutely necesarry, since the observation points are randomised every epoch and provide enough information for the model to guess the initial conditions. However since we explicitly know the initial conditions, adding them to the loss function is never a bad idea.

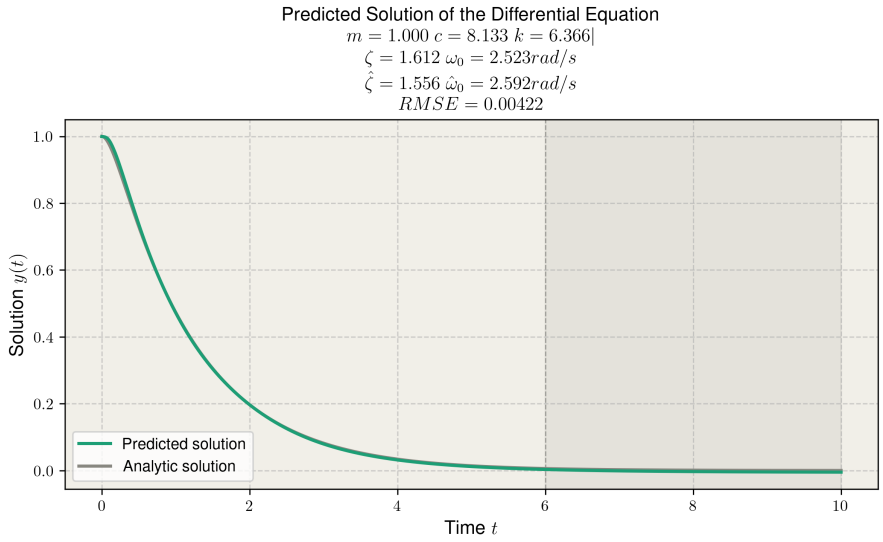


Figure 2.22: Predicted solution and parameters for an overdamped harmonic oscillator with randomised parameters.

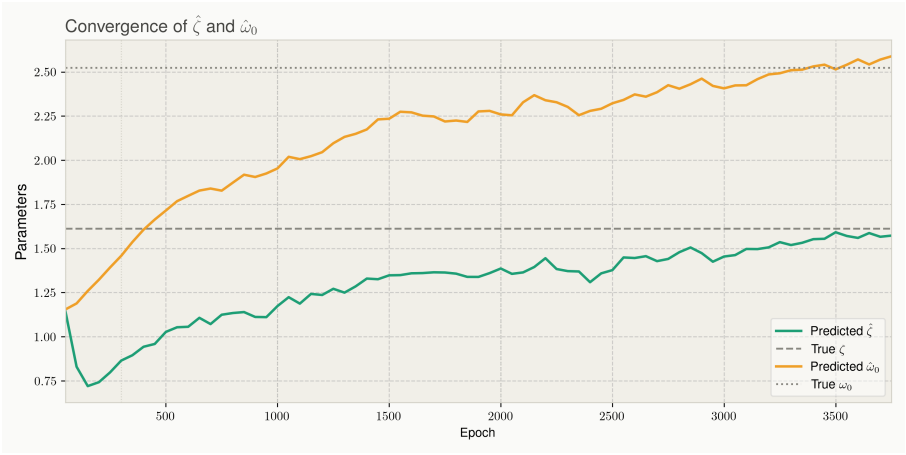


Figure 2.23: Parameter convergence to true values for an overdamped harmonic oscillator with randomised parameters.

We can see that the model is able to recover the equation parameters with great acuraccy for the overdamped case. This is quite impressive given that the analytic solutions are not very different for varying values of ζ and ω_0 for the overdamped case. Tis will be discussed in more detail in section 2.4.4.

2.4.2 Critically damped case

For the critically damped demonstration example, ζ can obviously not be randomised since it has to be equal to 1 by definition. The true and predicted parameters are:

Table 2.9: Overdamped parameters

Parameter	True value	Predicted value
ζ	1.000	1.014
ω_0	$4.000 rad/s$	$4.350 rad/s$

Figures 2.24 and 2.25 show the predicted solution and parameter convergence to the true values.

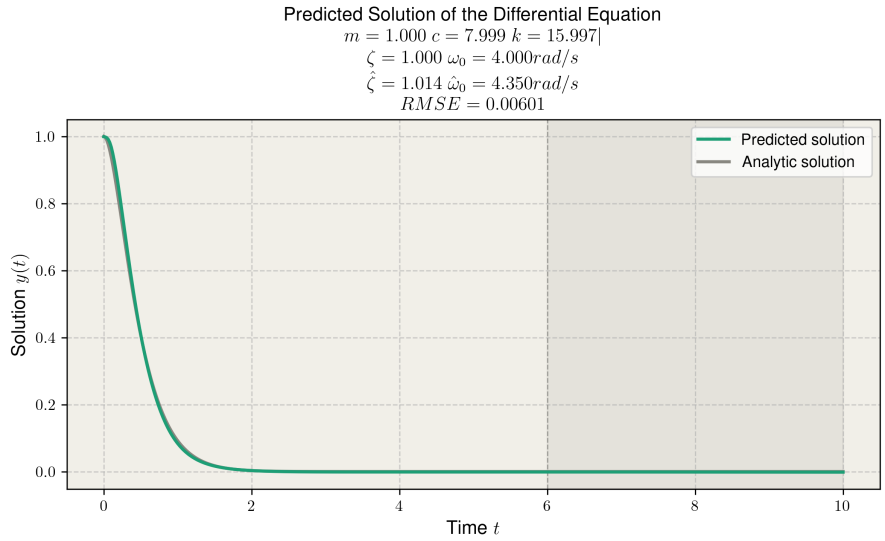


Figure 2.24: Predicted solution and parameters for a critically damped harmonic oscillator with randomised parameters.

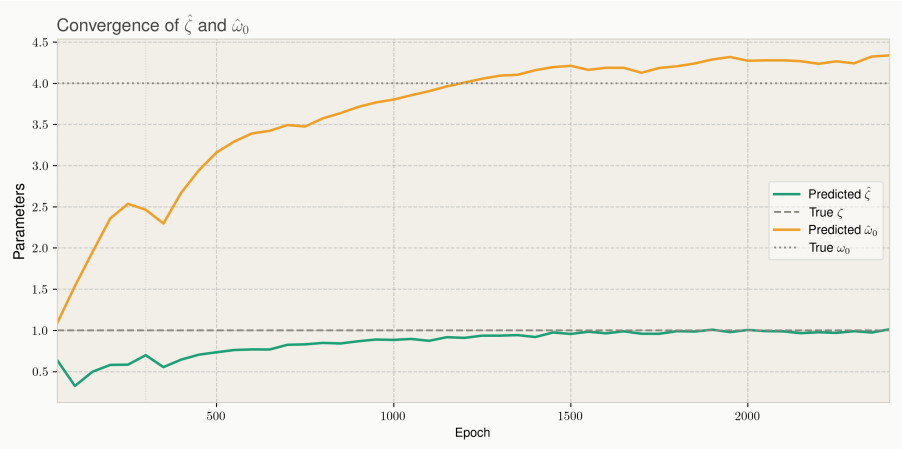


Figure 2.25: Parameter convergence to true values for a critically damped harmonic oscillator with randomised parameters.

We can see that the model is able to recover the equation parameters with relatively good acuraccy for the critically damped case. Again, this case is very similar to the overdamped case so comparable results are expected.

2.4.3 Underdamped case

For the overdamped demonstration example, the randomised and predicted parameters are:

Table 2.10: Overdamped parameters

Parameter	True value	Predicted value
ζ	0.160	0.171
ω_0	$1.950 rad/s$	$1.961 rad/s$

Figures 2.26 and 2.27 show the predicted solution and parameter convergence to the true values.

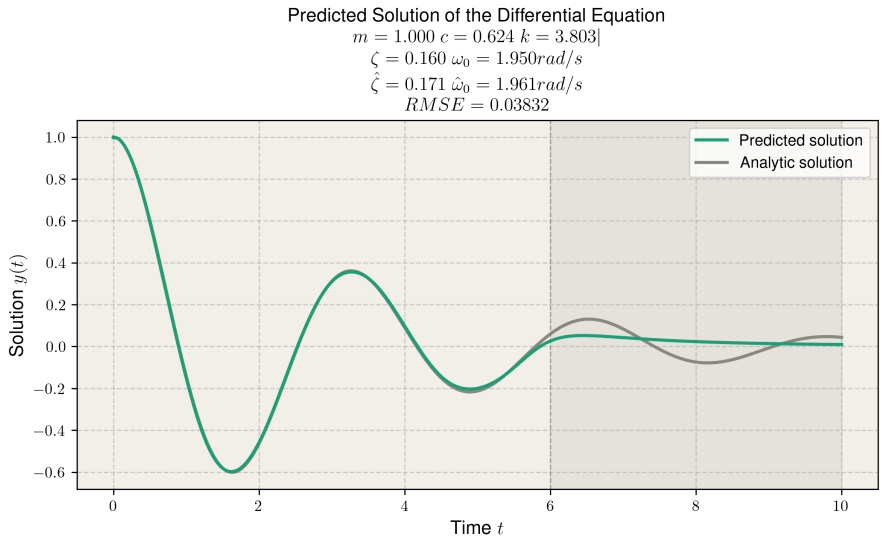


Figure 2.26: Predicted solution and parameters for an underdamped harmonic oscillator with randomised parameters.

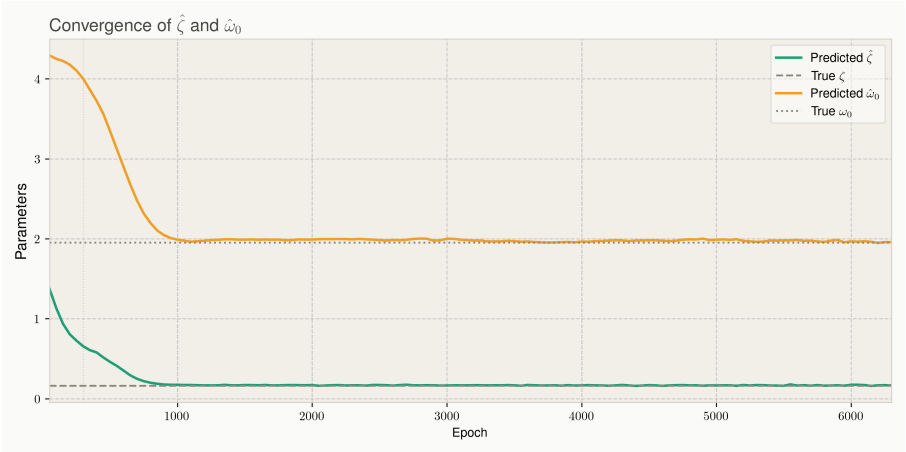


Figure 2.27: Parameter convergence to true values for an underdamped harmonic oscillator with randomised parameters.

We can see that the model is able to recover the equation parameters with great acuraccy for the underdamped case. The analytic solutions for this case are more dependent on the equation parameters than the previous two cases, so it is to be expected that the model can extract the parameters with the same or better acuraccy.

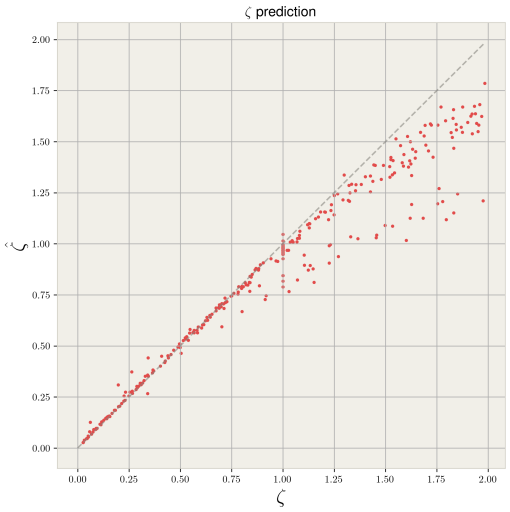
2.4.4 Statistical analysis

The previous three demonstration examples are a good illustration of the capabilities of the inverse model, but are not enough to make well-reasoned and statistically backed conlcusions about the model acuraccy. For this, we need a more quantifiable way to gauge the accuracy of the model.

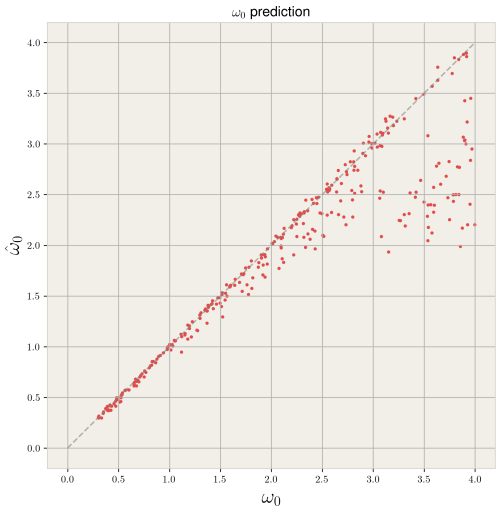
One of the sources of uncertainty when estimating the parameters is the noise applied to the training observation points. The training points are randomised every epoch and should, in principle, not have

much influence on the exact prediction, but it makes the process non-deterministic nonetheless. Furthermore, up until now we have only tested the parameter extraction on 3 $(\zeta,\ \omega_0)$ pairs. This is not yet representative for the model performance over a wide range of valeus for these parameters.

In order to account for both of these effects, we can run a large amount of parameter estimations with randomised equation parameters for every different model. This will both give an idea of the variance of the estimations due to the noisy training data, and generalise the three demonstration examples to a larger domain of $(\zeta,\ \omega_0)$ values. I have chosen to first randomize the damped case, with a 45% chance of under- or overdamped and a 10% chance of critically damped. Then, $(\zeta,\ \omega_0)$ is randomised and the inverse model is trained to estimate the parameters. In the end, 304 total estimations were performed. Figures 2.28a and 2.28b show the true-predicted parameter values.



(a) Scatter plot of 304 true and predicted ζ pairs.



(b) Scatter plot of 304 true and predicted ω_0 pairs.

Figure 2.28: Scatter plots of true and predicted physical parameters for the damped harmonic oscillator.

AS we can see, the estimations are quite good in general. For small parameter values, the model is very good at precisely extracting the true values. For larger values however, the model seems to underestimate the parameter values. To quantify this further, we can calulate the *RMSE* of the two parameters in the three different cases. This leads to the following results:

Table 2.11: Parameter estimation RMSE across different cases

Regime	**RMSE** ζ	**RMSE** ω_0
Full domain	0.1732	0.4791
Underdamped	0.0351	0.2960
Critically damped	0.0672	0.4007
Overdamped	0.2649	0.6415

The *RSME* indeed increases for the critically and overdamped cases. In order to also take the effect of ω_0 into account, we can plot the *parameter space*, where the axes represent ζ and ω_0. Figure 2.28b shows the $(\zeta,\ \omega_0)$ in the parameter space, color coded by the *RMSE* of the predicted values.

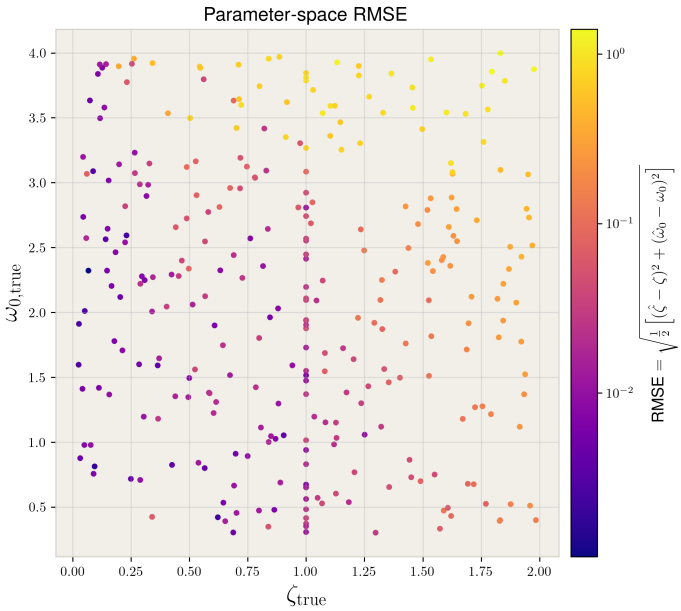
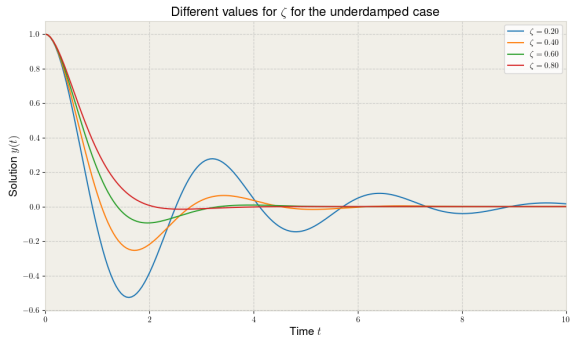
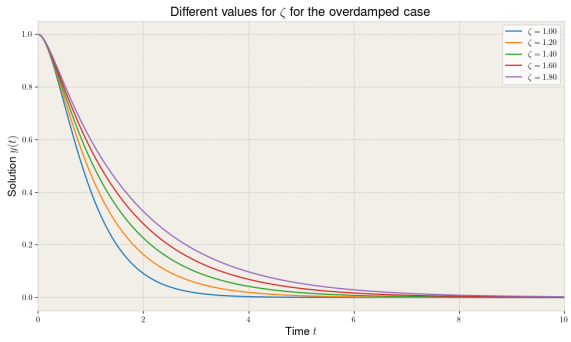


Figure 2.29: Parameter space of 304 true (ζ, ω_0) pairs and the *RMSE* compared to the predicted values.

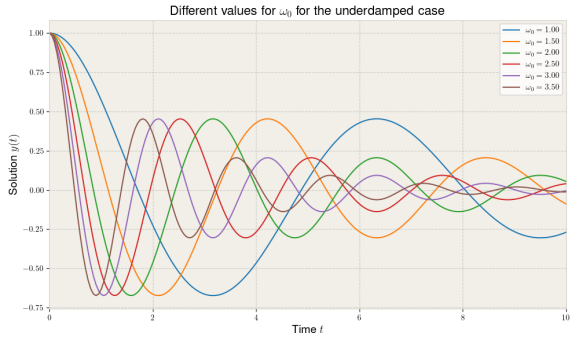
Again, we can see that large parameter values generally lead to a larger *RMSE*. One possible explanation is that the parameters become harder to extract because the soltuions to the ODE becone less and less dependent on the parameters. This is illustrated in the Figure 2.30, where the solutions to the ODE are plotted for different values of ζ and ω_0 in the underdamped and overdamped regime.



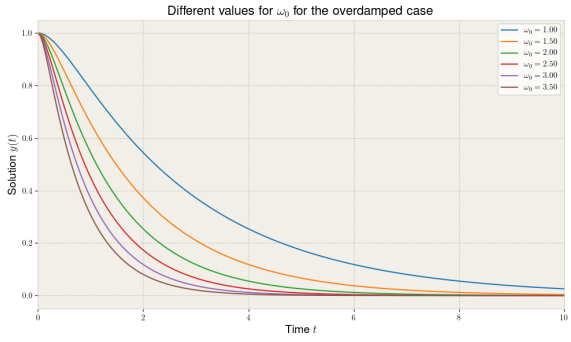
(a) Underdamped solutions for different ζ



(b) Overdamped solutions for different ζ



(c) Underdamped solutions for different ω_0



(d) Overdamped solutions for different ω_0

Figure 2.30

We can observe a lot more dispersion for the underdamped case, which means that the uncertainty of the estimates of the parameters will be smaller in this regime. For the overdamped case, the solutions are similar, and it is more difficult to extract the correct parameter value.

We can conclude that the most important source of uncertainty when estimating the equation parameters is the noise applied to the training data. We can investigate the uncertainty through a statistical approach where we estimate the parameters for a large amount of randomised equations, which also clarifies the model acuraccy in the different damped regimes. We conclude that the higher the damping, the more difficult the parameter extraction becomes since the solutions look very similar for variations in the parameters.

3 Burger's Equation

In this chapter, we will apply PINNs to solve the 1D Burger's equation, a PDE which describes convection and diffusion. The code for this chapter can be found in the `Burgers_Equation` directory. The general form of the inviscid Burger's equation is given by:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0 \tag{3.1}$$

This equation is sometimes written in the following notation:

$$u_t + uu_x = 0 \tag{3.2}$$

This is a hyperbolic first-orde PDE, which can be solved using the method of characteristics (see section3.1.1).

This equation occurs in fluid dynamics, where the field $u(x,\ t)$ represents the velocity of the fluid. In our case, we are then studying a one-dimensional fluid. The term u_t represents the rate of change in the velocity field. It is equal (if the viscosity is negligent) to the advection term uu_x, which gives a wave-like equation where the speed of the transported quantity u is proportional to the value of u itself [9]. This makes sense in the fluid dynamical interpretation of the equation; if interactions between the particles in the fluid can be ignored, and the velocity of each particle stays constant, the transport of the velocity is proportional to the velocity itself.

The fact that the transport speed scales with u leads to interesting behavior. It means that the top of a wave will travel faster than the bottom of the wave, causing it to fold over on itself. This leads to the creation of a discontinuïty, which is called a *shockwave*. On the other hand, when the gradient of u is positive, it can lead to the top of a wave 'running away' from its base, leading to the velocity field being stretched out and the gradient decreasing. This is called a *rarefaction*.

To avoid discontinuïties in the solution, which will lead to numerical instability, we can add a viscosity term $\nu\frac{\partial^2 u}{\partial x^2}$ which will prevent the formation of sharp gradients. In the fluid dynamical interpretation of the equation, this term represents the internal friction between the particles in the fluid. The viscid Burger's equation is given by:

$$u_t + uu_x = \nu u_{xx} \tag{3.3}$$

where ν iq called the *kinematic viscosity*. This equation is now not hyperbolic, but parabolic. It cannot be solved using the method of characteristics and needs an initial condition $u(x,\ 0)$ and boundary conditions $u(0,\ t)$ and $u(L,\ t)$ to have a unique, well-defined solution. This is the equation we will attempt to solve using a PINN.

3.1 Ground truth solving strategies

In order to train a machine learning model to solve the equation, we need ground truth data to use for the model training. Since we are not doing a real-world experiment, we do not have access to real data and have to simulate the ground truth data ourselves. In order to do this, there are multiple possible solving strategies.

3.1.1 Method of characteristics

As explained before, the Method Of Characteristics (MOC) can only be used for the inviscid equation. MOC works by looking for $(x(s),t(s))$ curves along which teh PDE collapsed to an ODE for the parameter s. Consider such a curve, then the chain rule tells us that:

$$\frac{du}{ds} = u_t \frac{dt}{ds} + u_x \frac{dx}{ds} \tag{3.4}$$

Comparing this to the inviscid Burger's equation, we can equate the coefficients of u_x and u_t:

$$\begin{aligned} \frac{dt}{ds} &= 1 \rightarrow t = s \\ \frac{dx}{ds} &= u \rightarrow \frac{dx}{dt} = u \\ \frac{du}{ds} &= 0 \rightarrow u(s) = u_0(x) \end{aligned} \tag{3.5}$$

This implies that the solution u remains constant along curves $x(t) = u_0(x)t + x_0$. This can be used to solve the equation, as shown on Figures 3.1 and 3.2.

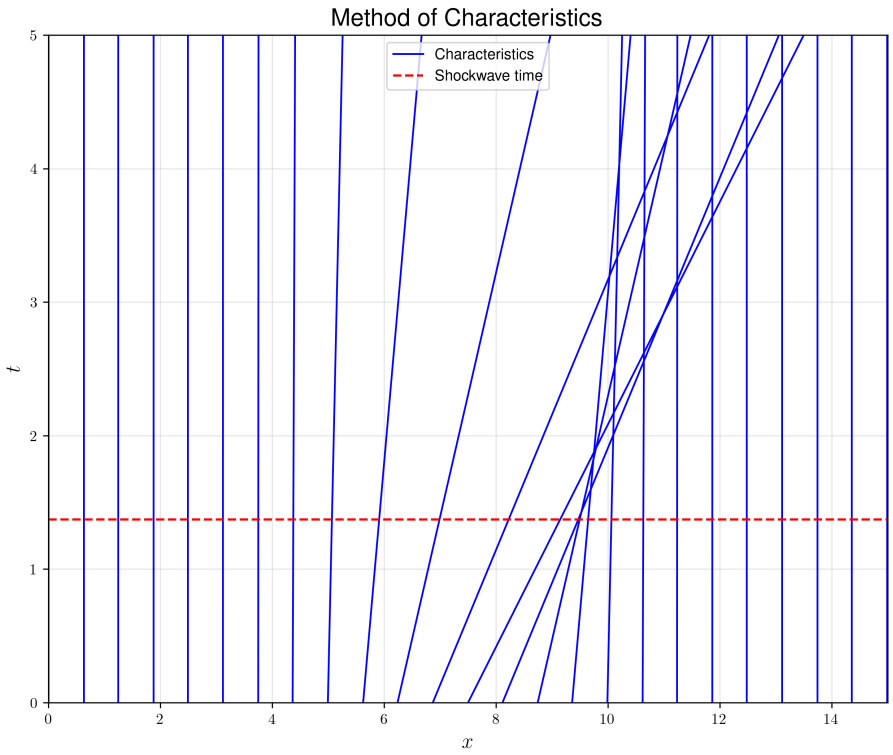
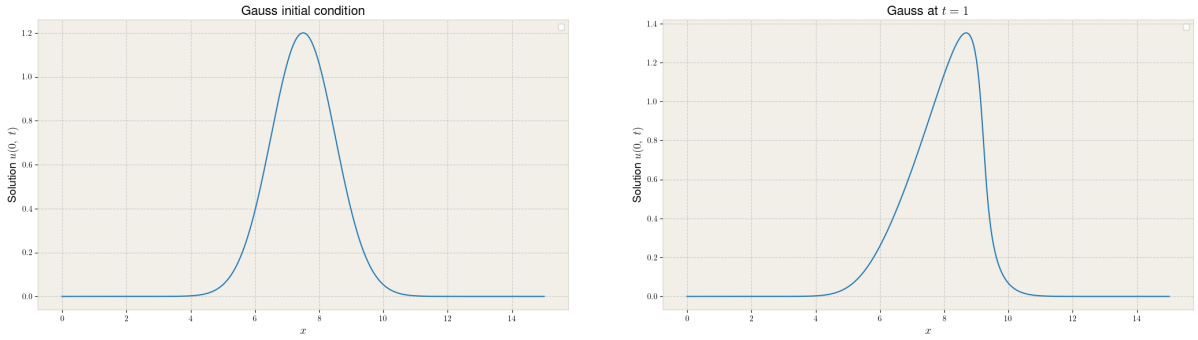


Figure 3.1: Method of characteristics applied to a Gaussian initial condition.



(a) Initial Gauss condition.

(b) Gauss at $t = 1$.

Figure 3.2: Gaussian initial condition and its time evolution.

This is a powerful method to predict when a shock will occur; a shockwave takes place when two characteristics cross, which is equal to $t = -\frac{1}{\min_x u_x(x,\, t=0)}$. However, we will be concerned with the viscid equation. To get ground truth solutions, we must look at other solving algorithms.

3.1.2 Numerical PDE solvers

The first option is to implement numerical iterative methods, which use finite-differences to discretize the derivatives in the equation. The solution is then calculated by dividing the releavnt domain into a grid with spacings Δx and Δt, and continously solving for the soltuion at the next Δt. I implemented two different discretisation schemes:

3.1.2.1 Euler's method (upwind)

Using Euler's method with upwind advection, the scheme is first order in time and first order in space for the advection term. The diffusion term is handled with second-order central differences. The discretisation scheme becomes:

$$u_i^{n+1} = u_i^n - \Delta t\, A_i^n + \Delta t\, \nu \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} \tag{3.6}$$

where the upwind advection term A_i^n is chosen based on the sign of u_i^n:

$$A_i^n = \begin{cases} u_i^n \dfrac{u_i^n - u_{i-1}^n}{\Delta x} & \text{if } u_i^n \geq 0 \quad \text{(backward difference)} \\ u_i^n \dfrac{u_{i+1}^n - u_i^n}{\Delta x} & \text{if } u_i^n < 0 \quad \text{(forward difference)} \end{cases} \tag{3.7}$$

The upwind scheme selects the finite difference direction based on the local flow direction, which improves numerical stability by respecting the characteristic direction of the advection term.

3.1.2.2 Lax-Wendroff scheme

Using the Lax-Wendroff scheme, which is second order in both time and space (both diffusion and advection), the discretisation scheme becomes:

$$u_i^{n+1} = u_i^n - \frac{\Delta t}{\Delta x}\left(F_{i+\frac{1}{2}}^n - F_{i-\frac{1}{2}}^n\right) + \Delta t\, \nu \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} \tag{3.8}$$

with the Burgers flux $f_i^n = \frac{1}{2}(u_i^n)^2$ and local wave speed $c_{i+\frac{1}{2}}^n = \frac{1}{2}(u_i^n + u_{i+1}^n)$. The diffusion term is handled separately with second-order central differences. The Lax-Wendroff scheme reduces numerical diffusion compared to the upwind Euler method, at the cost of introducing small oscillations near sharp

gradients.

Numerical schemes can be a powerful method to calculate the ground truth, but we have no good way of validating the accuracy of the result. Numerical errors will propagate and increase with time, depending on the accuracy of the discretisation scheme. Calculating the physics residual using finite differences would be a circular reasoning, since the solutions were constructed using the very same discretisation; we would simply be reversing the calculations that have alreade been done, which gives a very low residual. This means that it is difficult to be sure of the accuracy of the ground truth. Therefore, I have chosen a different methodology.

3.1.3 Cole-Hopf Transformation

The viscid Burger's equation can be solved by applying the *Cole-Hopf Transformation*:

$$u = -2\nu \frac{\phi_x}{\phi} \tag{3.9}$$

Through a lengthy calculation [12], we find that the Burger's equation is transformed into the heat equation:

$$\phi_t = \nu \phi_{xx} \tag{3.10}$$

This equation has a closed-form solution once the initial condition $\phi(x,t=0) = \phi_0(x)$ is defined by convoluting with the heat kernel:

$$\phi(x,t) = \frac{1}{\sqrt{4\pi\nu t}} \int_{-\infty}^{\infty} \phi_0(x') \exp\left(-\frac{(x-x')^2}{4\nu t}\right) dx'. \tag{3.11}$$

With this solution, we can recover $u(x,\ t)$ for ant value of t. For some initial conditions, analytic solutions to the Burger's equation can be found using this method. For more flexibility, I have chosen to solve the convolution integral numerically, so that any initial condition can be used as a ground truth.

It is obviously impossible to evaluate the convolution integral from $-\infty$ to $+\infty$ numerically. However, if we constrain the domain of the integral to $[0, L]$ where L is the length of our system, boundary effects will occur since the heat kernel will be cut off at the edges. To fix this, we pad the sides of the initial condition with a constant function matching the Dirichlet boundary condition so the domain is extrapolated. We then compute the ground truth solution using the Carl-Hopf method, and remove the paddings. In this way, we can also apply Dirichlet boundary conditions at a finite interval.

For validation, it is wise to calculate the *RMSE* of the physics residual. For a gaussian initial condition with $\nu = 0.01$ and $t_end = 10$, this yields:

$$RMSE = 5.19 \cdot 10^{-6}$$

This is a very small residual, which confirms the transformation can give us accurate solutions of the Burger's equation.

Using this method, I calculate the values for $u(x,\ t)$ on a dense grid of points. In order to randomise the data generation even more and avoid fine tuning to the specific spacings of this grid, I will make noisy data points by making a bilinear interpolation of the ground truth solution grid.

3.2 PINNs for the Burger's equation

With the ground truth data simulation method chosen, we can now train PINNs to solve the viscid Burger's equation. Just like for the damped harmonic oscillator, I will discuss the code implementation and show some examples, study the different loss terms and hyperparameters, and finally attempt to solve the inverse problem.

3.2.1 Code implementation

The implementation of the neural network is strongly based on the implementation of the damped harmonic oscillator PINN. The same improvements I have previously discussed apply; randomised training data and collocation points, early stopping, and model saving have all been implemented. The most important change is obviously the model itself, which now takes two inputs (x,t) instead of one input (t). This means that the loss functions and plotting methods also need to be adapted to handle two dimensional data. Some more adaptations that require an bit more in-depth explanation are included below.

3.2.1.1 File structure

The structure of the code is roughly the same as for the damped harmonic oscillator:

Burger's equation file structure

```
Burgers_Equation/
|-- config.py          - dataclass holding all physical constants,
|                       hyperparameters, runtime settings
|-- analytic.py        - ground truth solution schemes, including
|                       upwind euler, lax-wendroff and Cole-Hopf method
|-- data.py            - data generation based on ground truth solutions
|                       for collocation points, observation points,
|                       and initial conditions
|-- model.py           - FCNet architecture and predictor functions
|-- losses.py          - loss functions for the training of the model
|-- trainer.py         - training loop
|-- plot.py            - all plotting functions
|-- utils.py           - helper file for small functions
```

The only notable difference compared to the damped harmonic oscillator is the inclusion of the `analytic.py` file. The ground truth is now much more complex to calculate than for the damped harmonic oscillator, so I have chosen to do all the ground truth solution generation in a separate file.

3.2.1.2 Boundary condition loss

We introduce a new loss term: $\mathcal{L}_{b.c.}$ to force the PINN to respect the boundary conditions of the system. Every epoch, 50 boundary condition points are chosen randomly along both boundaries of the system (at $x = 0$ and $x = L$) and the predicted value $\hat{y}$ and derivative $d\hat{y}$ are evaluated (using `autograd`). The loss term $\mathcal{L}_{b.c.}$ and total loss are then calculated as:

$$\begin{aligned} \mathcal{L}_{b.c.} &= \frac{1}{N}\sum_{i=1}^{N}\left[(y_{i,\,left} - \hat{y}_{i,\,left})^2 + (y_{i,\,right} - \hat{y}_{i,\,right})^2\right] \\ \mathcal{L}_{tot} &= \mathcal{L}_{data} + \lambda_{i.c.}\mathcal{L}_{i.c.} + \lambda_{b.c.}\mathcal{L}_{b.c.} + \lambda_{phys}\mathcal{L}_{phys} \end{aligned} \tag{3.12}$$

3.2.1.3 Plotting methods

I ahve implemented a wide range of plotting methods, which are described below:

-

3.2.2 Code demonstration

3.2.3 Comparison between blind and physics informed extrapolation

3.2.4 Comparison with traditional ML model

3.2.5 Hyperparameter optimisation

3.3 Inverse problem

3.3.1 Underdamped case

3.3.2 Critically damped case

3.3.3 Overdamped case

3.3.4 Statistical analysis

4 Conclusion

Bibliography

[1] GeeksforGeeks. Loss function for linear regression in machine learning, 2024. Accessed: 2026-05-06. 7

[2] GeeksforGeeks. Adam optimizer, 2025. Accessed: 2026-05-07. 8

[3] GeeksforGeeks. Momentum-based gradient optimizer, 2025. Accessed: 2026-05-07. 8

[4] GeeksforGeeks. Rmsprop optimizer in deep learning, 2025. Accessed: 2026-05-07. 9

[5] GeeksforGeeks. Hyperparameter tuning, 2026. Accessed: 2026-05-07. 8

[6] Y. Guo, Z. Fu, J. Min, S. Lin, X. Liu, Y. F. Rashed, and X. Zhuang. Long-term simulation of physical and mechanical behaviors using curriculum-transfer-learning based physics-informed neural networks. 2025. Accessed: 2026-05-07. 8

[7] PyTorch Contributors. *PyTorch Documentation, Version 2.11*. PyTorch, 2025. Accessed: 2026-05-12. 12, 13

[8] Stack Overflow contributors. What does `unsqueeze` do in PyTorch?, 2019. Accessed: 2026-05-12. 14

[9] N. Van Remortel and H. Einsle. Exam june term 2026: Using a physics informed neural network to solve a pde. Course exam assignment, Programmeren voor fysici, May 2026. Submission deadline: May 22, 2026. 37

[10] G. van Rossum, J. Lehtosalo, and Ł. Langa. PEP 484 – type hints. Python Enhancement Proposal 484, Python Software Foundation, 2015. Status: Final. Python version: 3.5. 11

[11] S. Watanabe. Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance, 2025. 24, 28

[12] G. B. Whitham. *Linear and Nonlinear Waves*. Pure and Applied Mathematics: A Wiley Series of Texts, Monographs and Tracts. John Wiley & Sons, New York, 2011. 40